Horizon Protocol

A Yield Tokenization Protocol for Starknet

Technical Specification | Version 1.0

December 2025

Alex Metelli

https://github.com/ametel01/horizon-starknet

License: BSL-1.1 (converts to GPL-3.0 on 2028-12-19)

Abstract

Horizon Protocol is a yield tokenization system deployed on Starknet that decomposes yield-bearing assets into separate Principal Tokens (PT) and Yield Tokens (YT). This separation enables three distinct financial strategies: fixed-yield positions through discounted PT acquisition, leveraged yield exposure via YT trading, and liquidity provision in the protocol's native PT/SY automated market maker.

The protocol operates on a fundamental conservation invariant: one unit of Standardized Yield token (SY) can be split into one PT and one YT before maturity, and this process is reversible. At maturity, each PT is redeemable for exactly one unit of the underlying SY, providing a guaranteed redemption value. YT holders receive all yield generated by the underlying asset until expiry, after which the token expires worthless.

Horizon employs a logit-based AMM curve specifically designed for time-decaying assets, ensuring PT price converges to 1 as maturity approaches. This specification provides formal definitions, mathematical proofs, economic analysis, and comprehensive risk assessment suitable for auditors, researchers, developers, and investors.

Keywords: yield tokenization, principal tokens, yield tokens, Starknet, DeFi, automated market makers, fixed-point arithmetic, smart contracts, protocol specification, mathematical proofs

Contents

Abstract **2**

1 Motivation & Problem Statement **10**
1.1 Yield Variability and Rate Uncertainty 10
1.2 Why Existing Markets Do Not Solve Rate Certainty 11
1.2.1 Traditional Lending Protocols 11
1.2.2 Fixed-Rate Lending Protocols 11
1.2.3 Yield Tokenization as a Decomposition Primitive 11
1.3 Starknet Constraints and Design Opportunities 12
1.3.1 Constraints 12
1.3.2 Integration with Starknet Yield Sources 12
1.3.3 Oracle Integration 13
1.4 Assumptions and Threats to Guarantees 13

2 System Overview **14**
2.1 Protocol Architecture 14
2.1.1 Layer Model 14
2.2 Token Flow 14
2.2.1 Forward Flow (Deposit and Split) 14
2.2.2 Reverse Flow (Redemption) 14
2.3 Token System 15
2.4 Core Protocol Invariants 15

3 Formal Model & Notation **16**
3.1 State Variables 16
3.1.1 Market State 16
3.1.2 Time State 16
3.1.3 Index State 17
3.2 Constants 17
3.3 Fixed-Point Arithmetic 17
3.3.1 WAD Arithmetic (Interface Layer) 17
3.3.2 cubit 64.64 (Internal Layer) 18

3.4 Time Model . . . 19
3.5 Core Equations . . . 19
3.5.1 Pool Proportion and Logit Transformation . . . 19
3.5.2 Pricing Model . . . 20
3.5.3 Rate Anchor Dynamics . . . 21
3.5.4 Fee Model . . . 21
3.5.5 Yield Index Model . . . 21
3.6 Invariants . . . 22

4 Core Mechanism Design 23
4.1 SY Token Mechanism . . . 23
4.1.1 Deposit . . . 24
4.1.2 Redeem . . . 25
4.1.3 Exchange Rate (Index) . . . 25
4.2 PT/YT Minting Mechanism . . . 26
4.2.1 Minting Authority Invariant . . . 26
4.2.2 Index Update Triggers . . . 26
4.2.3 Mint PY (Split SY to PT + YT) . . . 27
4.2.4 Interest Accrual . . . 28
4.3 Redemption Mechanisms . . . 29
4.3.1 Pre-Expiry Redemption (PT + YT to SY) . . . 29
4.3.2 Post-Expiry Redemption (PT to SY) . . . 30
4.3.3 YT Interest Claim . . . 31
4.4 AMM Mechanism . . . 31
4.4.1 Implied Rate Storage . . . 31
4.4.2 Precompute Values . . . 32
4.4.3 Price Calculation . . . 32
4.5 Swap Mechanics . . . 32
4.5.1 Swap Exact PT for SY . . . 33
4.5.2 Swap Exact SY for PT . . . 35
4.5.3 Inverse Swaps . . . 36
4.6 Liquidity Provision . . . 38
4.6.1 Add Liquidity (Mint LP) . . . 38
4.6.2 Remove Liquidity (Burn LP) . . . 39
4.7 Key Invariants . . . 39
4.7.1 Safety Invariants (Enforced by Code) . . . 39
4.7.2 Economic Invariants (Maintained by Arbitrage) . . . 40
4.8 Failure Modes and Recovery . . . 40
4.8.1 Oracle Failure . . . 41
4.8.2 Underlying Depeg . . . 41

4.8.3 Pool Imbalance . . . 41
4.8.4 Flash Loan Attacks . . . 41
4.8.5 Expired Market Operations . . . 42

5 Economic Analysis & Incentives 43
5.1 Actors and Incentives . . . 43
5.1.1 PT Holders (Fixed-Rate Exposure) . . . 43
5.1.2 YT Holders (Yield Speculation) . . . 44
5.1.3 Liquidity Providers . . . 45
5.1.4 Arbitrageurs . . . 45
5.2 Equilibrium Conditions . . . 46
5.2.1 Price Equilibrium . . . 46
5.2.2 Liquidity Equilibrium . . . 46
5.3 Stress Scenarios . . . 47
5.3.1 Yield Collapse . . . 47
5.3.2 Yield Spike . . . 47
5.3.3 Mass Redemption . . . 47
5.3.4 Underlying Depeg . . . 48
5.4 Mechanism Properties . . . 48
5.4.1 Equilibrium Characterization . . . 48
5.4.2 Incentive Properties . . . 49
5.5 Fee Economics . . . 49
5.5.1 Fee Retention Model . . . 49
5.5.2 Optimal Fee Rate . . . 49
5.6 Value Accrual . . . 50
5.6.1 Value Flow . . . 50
5.6.2 Protocol Value Capture . . . 50

6 Risk Analysis & Threat Model 51
6.1 Smart Contract Risk . . . 52
6.1.1 Code Vulnerabilities . . . 52
6.1.2 Dependency Risk . . . 52
6.2 Upgrade Risk . . . 53
6.2.1 Upgrade Impact by Contract . . . 53
6.2.2 Upgrade Policy . . . 53
6.2.3 User Protections During Upgrades . . . 54
6.3 Oracle Risk . . . 54
6.3.1 Oracle Parameters . . . 54
6.3.2 Oracle Failure Behavior . . . 55
6.3.3 Exchange Rate Manipulation . . . 55

6.4 Market Risk . . . 56
6.4.1 Liquidity Risk . . . 57
6.4.2 Impermanent Loss Risk . . . 57
6.5 Underlying Asset Risk . . . 57
6.6 Governance and Admin Risk . . . 58
6.6.1 Admin Key Compromise . . . 58
6.6.2 Malicious Upgrade Detection . . . 59
6.6.3 Role Escalation . . . 59
6.7 Starknet Infrastructure Risk . . . 59
6.8 User Operational Risk . . . 60
6.9 Threat Model Summary . . . 60
6.9.1 Risk Priority Matrix . . . 60
6.10 Exclusions: What the Protocol Does NOT Protect Against . . . 61

7 Governance & Upgrade Model 63
7.1 Contract Classification . . . 63
7.1.1 Internal Contracts (Protocol-Controlled) . . . 64
7.1.2 External Dependencies (Third-Party) . . . 64
7.2 Admin Powers . . . 65
7.2.1 Powers Available on All Upgradeable Contracts . . . 65
7.2.2 Contract-Specific Admin Functions . . . 65
7.3 Role-Based Access Control . . . 66
7.4 Emergency Procedures . . . 67
7.4.1 Emergency Powers . . . 67
7.4.2 Pause Scope . . . 67
7.4.3 Emergency Response Process . . . 68
7.5 Upgrade Process . . . 69
7.5.1 Standard Upgrade Steps . . . 69
7.5.2 Storage Compatibility Requirements . . . 70
7.6 Governance Attack Surface . . . 71
7.7 Trust Assumptions . . . 72
7.7.1 Trusted (Cannot Be Verified) . . . 72
7.7.2 Verifiable On-Chain . . . 72
7.7.3 Trustless (Given Current Code) . . . 72
7.8 Governance Roadmap . . . 73

8 Related Work 74
8.1 Pendle Finance (Design Lineage) . . . 74
8.1.1 Design Parity . . . 74
8.1.2 Execution Divergence . . . 75

8.2 Element Finance (Historical Precedent) 75
8.2.1 Element's Contribution 75
8.2.2 YieldSpace Limitations 75
8.3 Fixed-Rate Lending Protocols 76
8.4 Starknet Execution Environment 77
8.4.1 Protocol-Critical Properties 77
8.4.2 Incidental Benefits 78
8.5 Design References and Prior Work 78
8.6 Positioning Summary 79

9 Conclusion 80
9.1 Conclusion 80
9.2 Core Guarantees 80
9.3 Design Trade-offs 81
9.4 Out-of-Scope Enhancements 81
9.5 Specification Status 82

A Mathematical Proofs 83
A.1 PT Price Convergence 83
A.2 Conservation Invariant 83
A.3 No-Arbitrage Pricing Condition 84

B Numerical Examples 86
B.1 PT Purchase Flow 86
B.2 LP Deposit and Withdrawal 87
B.3 YT Interest Claim 88
B.4 Assumptions Summary 89

C Contract Interface Specifications 90
C.1 Interface Summary 90
C.2 SY Interface 90
C.3 PT Interface 92
C.4 YT Interface 92
C.5 Market Interface 94
C.6 Revert Conditions Summary 96

D Glossary 97
Glossary 97

1 Motivation & Problem Statement

This chapter establishes why yield tokenization is necessary, why existing solutions are inadequate, how tokenization addresses these gaps, and what Starknet-specific considerations shape the protocol's design.

1.1 Yield Variability and Rate Uncertainty

Yield-bearing tokens have become fundamental primitives in decentralized finance. Liquid staking derivatives (nstSTRK, sSTRK), wrapped staking tokens (wstETH), and lending protocol receipt tokens generate yield through staking rewards, lending interest, or protocol revenue distribution. However, these yields share a common characteristic: **variability**.

In practice, the same position can swing from low single-digit APY to double-digit APY over short horizons, driven by staking participation, protocol incentives, and utilization. For treasury management, this converts yield into an uncertain cash-flow stream rather than an allocatable budget line.

Consider a user holding nstSTRK (Nostra's staked STRK derivative). The yield depends on:

- Starknet's staking reward rate, which fluctuates with network participation
- Nostra's protocol fee structure, subject to governance changes
- Market conditions affecting utilization and demand

This variability creates three requirements that existing primitives do not satisfy:

1. **Forecastability**: Treasuries, DAOs, and institutional holders require predictable cash flows for budgeting and liability matching.
2. **Rate-locking without collateral inefficiency**: Capital providers need fixed-rate exposure without overcollateralized debt positions or idle margin.
3. **A market for yield views and hedging**: Users with convictions about future yield direction require an efficient mechanism to express these views or hedge existing exposure.

1.2 Why Existing Markets Do Not Solve Rate Certainty

1.2.1 Traditional Lending Protocols

Protocols like Aave and Compound offer variable-rate lending and borrowing. While these markets provide yield, the rates fluctuate continuously based on utilization. A depositor earning 5% today may earn 2% tomorrow. These protocols solve capital efficiency but not rate certainty.

1.2.2 Fixed-Rate Lending Protocols

Protocols such as Notional Finance and Yield Protocol introduced fixed-rate borrowing and lending. However, these approaches face structural limitations:

Limitation	**Description**
Governance-driven term listing	Fixed maturities require protocol governance to launch new terms
Liquidity fragmentation	Capital splits across multiple maturity pools
Overcollateralization	Borrowers lock collateral; capital sits idle
Limited secondary market	Users lock rates but cannot trade rate exposure

1.2.3 Yield Tokenization as a Decomposition Primitive

Yield tokenization decomposes a yield-bearing position into two separable claims: a principal claim (PT) on redemption value at expiry, and a yield claim (YT) on all pre-expiry appreciation. This decomposition addresses the requirements introduced above without introducing borrower-side collateral constraints.

1. **Separation of Rights**: PT and YT can be held, traded, or used as collateral independently, enabling hedging and targeted exposure.
2. **Continuous Price Discovery**: Market prices for PT and YT encode collective expectations about future yield over the remaining time to expiry.

3. **Capital Efficiency**: Exposure is represented directly by transferable tokens rather than overcollateralized debt positions.
4. **Composability**: PT and YT implement the ERC-20 interface pattern on Starknet and integrate with existing DeFi tooling.
5. **Deterministic Maturity Handling**: PT mechanically converges to redemption value at expiry; settlement does not require term re-listing.

1.3 Starknet Constraints and Design Opportunities

Implementing yield tokenization on Starknet requires careful numeric design (felt-based arithmetic, u256 emulation) but also benefits from predictable execution and native integration with Starknet yield assets.

1.3.1 Constraints

Cairo, Starknet's native smart contract language, operates on a field element (felt252) primitive with different arithmetic properties than EVM's uint256. The protocol requires:

- Custom u256 handling for token amounts
- High-precision fixed-point math for rate calculations
- Explicit signed value handling for price differentials

The cubit 64.64 fixed-point library addresses these requirements, providing approximately 19 decimal digits of precision. All rate and price computations are defined in terms of deterministic fixed-point routines to avoid platform-dependent rounding.

1.3.2 Integration with Starknet Yield Sources

The protocol targets integration with Starknet's native yield-bearing tokens:

Token	Type	Yield Mechanism
nstSTRK	Liquid staking	Staked STRK + lending yield
sSTRK	Liquid staking	Staking rewards (derivative)
wstETH	Bridged staking	Ethereum staking rewards

Each integration requires an exchange rate oracle to track the underlying token's value appreciation.

1.3.3 Oracle Integration

Pragma is a major oracle provider on Starknet. For bridged assets, oracle feeds avoid per-operation cross-chain dependencies. The protocol uses Pragma's TWAP (time-weighted average price) feeds to mitigate single-block manipulation.

1.4 Assumptions and Threats to Guarantees

The protocol's guarantees depend on the following external assumptions. These are dependencies the protocol cannot enforce on-chain.

Assumption	**Failure Mode**
Underlying asset valuation is monotonic	Non-monotonic exchange rate breaks YT accounting
Oracle liveness and bounded error	Stale feeds cause mispricing or incorrect settlement
Sufficient arbitrage participation	Invariants not economically enforced during stress
Consistent timestamp semantics	Edge cases around expiry boundary
Underlying protocol solvency	SY value impairment

These are external dependencies; the protocol cannot enforce them on-chain. Users must evaluate these dependencies when interacting with the protocol.

2 System Overview

2.1 Protocol Architecture

The Horizon Protocol is organized into layered components with clearly defined responsibilities and upgrade boundaries. This separation isolates external dependencies from core accounting logic and limits the surface area affected by upgrades.

2.1.1 Layer Model

Layer	Components	Upgrade Scope
External	Underlying yield tokens, Pragma Oracle	Outside protocol control
Core	SY, PT, YT, Market	Upgradeable via governed proxy
Periphery	Router, Factory, MarketFactory	Upgradeable via governed proxy

2.2 Token Flow

The protocol implements a reversible asset flow centered around a deterministic split of standardized yield into principal and yield claims.

2.2.1 Forward Flow (Deposit and Split)

1. A user deposits an underlying yield-bearing token into the SY contract.
2. The SY contract mints a standardized yield token representing the deposited asset.
3. The user deposits SY into the YT contract, which atomically mints paired PT and YT tokens such that $1\,\mathrm{SY} = 1\,\mathrm{PT} + 1\,\mathrm{YT}$.

2.2.2 Reverse Flow (Redemption)

1. Before expiry, a user may redeem exactly one PT and one YT to recover one SY, reversing the split.
2. At expiry, PT redeems deterministically 1:1 for SY, while YT expires with zero value.
3. The user withdraws the underlying asset from the SY contract, receiving principal plus any accrued yield.

2.3 Token System

The protocol defines four token types representing distinct economic claims. All tokens implement the ERC-20 interface pattern and impose no transfer restrictions.

Token	Economic Role	Minting Mechanism	Transferability
SY	Standardized yield wrapper	Permissionless via deposit	Freely transferable
PT	Principal claim (redeemable at expiry)	Minted via YT contract	Freely transferable
YT	Yield claim (pre-expiry)	Minted via YT contract	Freely transferable
LP	Liquidity provider share	Minted by Market contract	Freely transferable

2.4 Core Protocol Invariants

The following invariants are safety-critical and must hold across all valid state transitions of the protocol.

1. **Principal Protection**: Each PT redeems deterministically for exactly one unit of SY at maturity.
2. **Conservation**: Prior to expiry, the combined value of PT and YT corresponds to one unit of SY.
3. **Price Convergence**: The AMM enforces monotonic convergence of PT price toward redemption value as expiry approaches.

4. **Yield Isolation**: All yield generated by the underlying asset accrues exclusively to YT holders prior to expiry.

3 Formal Model & Notation

This section formalizes the protocol's state, parameters, and mathematical invariants. All subsequent mechanism definitions rely on the notation and constants introduced here. We adopt a fixed-point arithmetic model (WAD scaling, 10^{18}) to ensure precision across all computations.

3.1 State Variables

The protocol state is partitioned into market state, time state, and index state. Each variable has a well-defined domain that is enforced by construction.

3.1.1 Market State

Symbol	Type	Domain	Description
S_{SY}	u256	$[0, 2^{256})$	SY reserve in market
S_{PT}	u256	$[0, 2^{256})$	PT reserve in market
L	u256	$[0, 2^{256})$	Total LP token supply
r	u256	$[0, 4.6 \cdot 10^{18}]$	ln(implied rate) in WAD
r_{root}	u256	$[0, 10^{20}]$	Scalar root parameter
r_{anchor}	u256	$[0, 10^{19}]$	Rate anchor parameter
ϕ	u256	$[0, 0.05 \cdot 10^{18}]$	Fee rate in WAD

Assumption: $S_{PT} + S_{SY} > 0$ at all times due to enforced minimum liquidity lock.

3.1.2 Time State

Symbol	Type	Domain	Description
T	u64	Unix timestamp	Expiry timestamp
t	u64	Unix timestamp	Current block timestamp
τ	u64	$[0, T]$	Time to expiry (seconds)
τ_y	u256	$[0, \infty)$	Time to expiry (years, WAD-scaled)

3.1.3 Index State

Symbol	Type	Domain	Description
I_{global}	u256	$[10^{18}, \infty)$	Global PY index (watermark)
I_{user}	u256	$[10^{18}, \infty)$	User's last recorded index

3.2 Constants

The protocol relies on the following global constants, which are fixed for the lifetime of a given market instance.

Scaling Constants:

- **WAD** ($W = 10^{18}$): Fixed-point scaling factor for all amounts and rates
- **Seconds per year** ($Y = 31{,}536{,}000$): Time normalization constant for annualized rates

Safety Bounds:

- **Minimum liquidity** ($L_{min} = 1000$): Locked LP to prevent division by zero
- **Max ln implied rate** ($r_{max} = 4.6 \cdot 10^{18}$): Caps implied rate at approximately 10,000% APY
- **Max proportion** ($p_{max} = 0.999 \cdot 10^{18}$): Prevents extreme pool imbalance
- **Min proportion** ($p_{min} = 0.001 \cdot 10^{18}$): Prevents extreme pool imbalance

3.3 Fixed-Point Arithmetic

The protocol uses a dual-layer precision model to balance gas efficiency with computational accuracy.

3.3.1 WAD Arithmetic (Interface Layer)

All token amounts and external-facing values use WAD scaling:

$$\text{wad_mul}(a, b) = \left\lfloor \frac{a \times b + W/2}{W} \right\rfloor$$

$$\text{wad_div}(a, b) = \left\lfloor \frac{a \times W + b/2}{b} \right\rfloor$$

where $W = 10^{18}$ and rounding is to nearest integer.

All intermediate multiplications are performed using widened precision to prevent overflow. Rounding is symmetric and biased toward the nearest integer to minimize cumulative drift.

3.3.2 cubit 64.64 (Internal Layer)

Transcendental functions use 64.64 binary fixed-point representation with 64 integer bits and 64 fractional bits, providing approximately 19 decimal digits of precision.

The 64.64 format represents a number x as:

$$x_{\text{fixed}} = x \times 2^{64}$$

Conversion Functions.

$$\text{wad_to_fp}(x) = \left\lfloor \frac{x \times 2^{64}}{10^{18}} \right\rfloor$$

$$\text{fp_to_wad}(x) = \left\lfloor \frac{x \times 10^{18}}{2^{64}} \right\rfloor$$

High-Precision Functions. The library provides the following transcendental functions:

- exp_wad(x): Exponential function, takes WAD input and returns WAD output
- exp_neg_wad(x): Negative exponential for decay functions
- ln_wad(x): Natural logarithm, returns (value, is_negative) tuple
- pow_wad(x, y): Power function computing x^y
- sqrt_wad(x): Square root function, WAD-normalized output

All transcendental functions clamp outputs on domain violation and return bounded values to preserve protocol safety.

3.4 Time Model

Time is measured in Unix timestamps (seconds since epoch):

$$\tau = T - t$$

where T is expiry timestamp and t is current block timestamp.

Time expressed in years (WAD-scaled):

$$\tau_y = \frac{\tau \times W}{Y}$$

For $t \geq T$, the protocol treats $\tau = 0$ and transitions to post-expiry redemption semantics.

3.5 Core Equations

This section defines the mathematical relationships governing AMM pricing, yield computation, and fee adjustment.

3.5.1 Pool Proportion and Logit Transformation

The proportion of PT in the pool determines market pricing:

$$p = \frac{S_{PT}}{S_{PT} + S_{SY}}$$

Domain constraint: $p_{min} \leq p \leq p_{max}$. Trades that would violate this constraint are rejected.

To map bounded pool proportions to an unbounded rate domain, the protocol uses a logit transformation:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

Properties:

- $\text{logit}(0.5) = 0$
- $\text{logit}(p) \to -\infty$ as $p \to 0$
- $\text{logit}(p) \to +\infty$ as $p \to 1$

3.5.2 Pricing Model

Time-to-expiry is incorporated through a scalar that dampens price sensitivity as maturity approaches.

Rate Scalar.

$$r_{scalar} = \frac{r_{root} \times Y}{\tau}$$

As $\tau \to 0$: $r_{scalar} \to \infty$, making prices insensitive to pool balance changes.

Exchange Rate. The PT/SY exchange rate derived from market state:

$$\text{exchangeRate} = \frac{\text{logit}(p)}{r_{scalar}} + r_{anchor}$$

Implied Rate. The annualized implied yield from the exchange rate:

$$r = \ln(\text{exchangeRate}) \times \frac{Y}{\tau}$$

Constraint: $0 \leq r \leq r_{max}$. Values outside this range are clamped.

PT Price. PT price in terms of SY, derived from implied rate:

$$P_{PT} = e^{-r \times \tau_y / W}$$

where τ_y is time to expiry in WAD-scaled years.

Properties:

- $P_{PT} < 1$ before expiry (trades at discount)
- $P_{PT} \to 1$ as $\tau \to 0$ (converges at maturity)

YT Price. YT price is derived directly from the conservation invariant:

$$P_{YT} = 1 - P_{PT}$$

Properties:

- $P_{YT} > 0$ before expiry
- $P_{YT} \to 0$ as $\tau \to 0$ (expires worthless)

3.5.3 Rate Anchor Dynamics

The rate anchor maintains implied rate continuity across trades, preventing discontinuous price jumps.

Anchor Calculation. Given the last stored implied rate r_{last} and current market state:

$$\text{exchangeRate}_{target} = e^{r_{last} \times \tau / Y}$$

$$r_{anchor} = \text{exchangeRate}_{target} - \frac{\text{logit}(p)}{r_{scalar}}$$

Anchor Update. After each trade, the anchor is recalculated from the new implied rate to ensure smooth price transitions.

3.5.4 Fee Model

Swap fees decay exponentially toward expiry, reducing friction as price convergence becomes more certain.

$$\phi_{adjusted} = e^{\ln(\phi_{root}) \times \tau / Y}$$

where ϕ_{root} is the base fee rate (e.g., 0.001 WAD = 0.1%).

Requirement: $0 < \phi_{root} < 1$ to ensure monotonic decay of swap fees.

Time to Expiry	**Adjusted Fee (if base = 0.1%)**
1 year	0.100%
6 months	$\approx$ 0.070%
1 month	$\approx$ 0.022%
1 week	$\approx$ 0.006%
At expiry	$\approx$ 0%

3.5.5 Yield Index Model

The PY index accounts for cumulative yield attributable to YT holders using a monotonic watermark mechanism.

Index Update.

$$I_{global} = \max(I_{global}, I_{current})$$

where $I_{current}$ is the current exchange rate from the oracle.

The watermark ensures I_{global} is monotonically non-decreasing. This prevents yield accounting errors during temporary rate decreases.

Interest Calculation. For a user with YT balance B_{YT} and last recorded index I_{user}:

$$\text{Interest} = B_{YT} \times \frac{I_{global} - I_{user}}{I_{user}}$$

After claiming, the user's index is updated: $I_{user} \leftarrow I_{global}$

3.6 Invariants

The following invariants are safety-critical. Any violation constitutes a protocol failure and must be prevented by construction or rejected at runtime.

Invariant	**Expression**	**Enforcement**
Supply equality	PT.supply = YT.supply	Contract logic
Conservation	PT + YT = SY (value)	Arbitrage
Index watermark	$I_{global}^{t+1} \geq I_{global}^{t}$	Contract logic
Proportion bounds	$p_{min} \leq p \leq p_{max}$	Trade rejection
Rate bounds	$0 \leq r \leq r_{max}$	Clamping
Reserve positivity	$S_{SY} > 0 \wedge S_{PT} > 0$	LP minimum lock

4 Core Mechanism Design

This section specifies the protocol's core operations with precise state transitions, revert conditions, and invariant preservation. Each mechanism is defined by its inputs, preconditions, state changes, events, returns, and revert conditions.

4.1 SY Token Mechanism

The Standardized Yield (SY) token wraps yield-bearing assets with a uniform interface. SY is a fixed 1:1 wrapper: one unit of underlying always mints one unit of SY, and one unit of SY always redeems one unit of underlying. The exchange rate tracked by the protocol is not the SY appreciation rate, but the underlying asset's index used for YT yield accounting.

Key distinction. SY itself does not appreciate. The `exchange_rate` (or `index`) tracks the underlying yield-bearing token's value growth and is used exclusively for calculating YT interest. This separation ensures SY remains a simple wrapper while yield accrual is handled by the YT mechanism.

4.1.1 Deposit

Algorithm 1: SY Deposit

function DEPOSIT($receiver, amount$) $\rightarrow shares$;

Input : $receiver$: address, $amount$: u256

Output : $shares$: u256 (amount of SY minted)

Preconditions : $receiver \neq 0$;

$amount > 0$;

`underlying.allowance`($caller, SY$) $\geq amount$;

`underlying.balanceOf`($caller$) $\leftarrow$ `underlying.balanceOf`($caller$) $- amount$;

`underlying.balanceOf`(SY) $\leftarrow$ `underlying.balanceOf`(SY) $+ amount$;

`SY.balanceOf`($receiver$) $\leftarrow$ `SY.balanceOf`($receiver$) $+ amount$;

`SY.totalSupply` $\leftarrow$ `SY.totalSupply` $+ amount$;

emit DEPOSIT($caller, receiver, amount, shares$);

return $shares \leftarrow amount$;

Reverts : $amount = 0$: "zero amount";

$receiver = 0$: "zero address";

allowance insufficient: "ERC20: insufficient allowance";

balance insufficient: "ERC20: transfer exceeds balance";

4.1.2 Redeem

Algorithm 2: SY Redeem

1 function REDEEM($receiver, amount$) $\rightarrow assets$;

Input : $receiver$: address, $amount$: u256

Output : $assets$: u256 (amount of underlying returned)

Preconditions : $receiver \neq 0$;

$amount > 0$;

`SY.balanceOf`($caller$) $\geq amount$;

2 `SY.balanceOf`($caller$) $\leftarrow$ `SY.balanceOf`($caller$) $- amount$;

3 `SY.totalSupply` $\leftarrow$ `SY.totalSupply` $- amount$;

4 `underlying.balanceOf`(SY) $\leftarrow$ `underlying.balanceOf`(SY) $- amount$;

5 `underlying.balanceOf`($receiver$) $\leftarrow$ `underlying.balanceOf`($receiver$) $+ amount$;

6 **emit** REDEEM($caller, receiver, amount, assets$);

7 **return** $assets \leftarrow amount$;

Reverts : $amount = 0$: "zero amount";

$receiver = 0$: "zero address";

balance insufficient: "ERC20: burn exceeds balance";

4.1.3 Exchange Rate (Index)

The exchange rate tracks the underlying asset's value appreciation and is used for YT yield accounting.

For ERC-4626 Vaults.

$$\texttt{exchange_rate} = \texttt{vault.convertToAssets}(W)$$

For Oracle-Based Tokens.

$$\texttt{exchange_rate} = \texttt{oracle.get_index}()$$

Invariant. The exchange rate is monotonically non-decreasing due to watermark protection. This is enforced by the global index update logic, not by the SY contract itself.

4.2 PT/YT Minting Mechanism

4.2.1 Minting Authority Invariant

Only the YT contract may mint or burn PT and YT tokens. This invariant is enforced by access control on the `mint` and `burn` functions of both token contracts.

Token	Authorized Caller	Enforcement
PT	YT contract only	`require`($caller$ = `yt_address`)
YT	YT contract only	Internal function (self-mint)

4.2.2 Index Update Triggers

The global index and user interest are updated at specific points to ensure correct yield accounting.

Function	Updates Global Index	Updates User Interest
`mint_py`	Yes	Yes (receiver)
`redeem_py`	Yes	Yes (caller)
`redeem_pt_after_expiry`	No	No
`claim_interest`	Yes	Yes (caller)
`YT.transfer`	Yes	Yes (sender and recipient)
`YT.transferFrom`	Yes	Yes (from and to)

YT Transfer Hook. YT transfers must update both sender and recipient interest before modifying balances. Failure to do so would cause interest accounting errors. This is implemented via an internal `_before_token_transfer` hook.

4.2.3 Mint PY (Split SY to PT + YT)

Algorithm 3: Mint Principal and Yield Tokens

function MintPY($receiver, amount_{sy}$) $\rightarrow (pt, yt)$;

Input : $receiver$: address, $amount_{sy}$: u256

Output : (pt, yt): amounts of PT and YT minted

Preconditions : `block.timestamp` $< expiry$;

$receiver \neq 0$;

$amount_{sy} > 0$;

`SY.allowance`$(caller, YT) \geq amount_{sy}$;

```
// Update indices before balance changes
```

$I_{global} \leftarrow \max(I_{global}$, `current_exchange_rate`$)$;

UpdateUserInterest($receiver$);

```
// Transfer SY from caller to YT contract
```

`SY.balanceOf`$(caller) \leftarrow$ `SY.balanceOf`$(caller) - amount_{sy}$;

`SY.balanceOf`$(YT) \leftarrow$ `SY.balanceOf`$(YT) + amount_{sy}$;

```
// Mint equal PT and YT to receiver
```

`PT.balanceOf`$(receiver) \leftarrow$ `PT.balanceOf`$(receiver) + amount_{sy}$;

`PT.totalSupply` $\leftarrow$ `PT.totalSupply` $+ amount_{sy}$;

`YT.balanceOf`$(receiver) \leftarrow$ `YT.balanceOf`$(receiver) + amount_{sy}$;

`YT.totalSupply` $\leftarrow$ `YT.totalSupply` $+ amount_{sy}$;

emit MintPY($caller, receiver, amount_{sy}, amount_{sy}$);

return $(amount_{sy}, amount_{sy})$;

Post-invariant: `PT.totalSupply` = `YT.totalSupply`;

Reverts : `block.timestamp` $\geq expiry$: "expired";

$amount_{sy} = 0$: "zero amount";

$receiver = 0$: "zero address";

allowance insufficient: "ERC20: insufficient allowance";

4.2.4 Interest Accrual

Algorithm 4: Update User Interest

```
function UpdateUserInterest(user);
Input          : user: address
Preconditions  : I_global has been updated to current value
if I_user[user] = 0 then
    I_user[user] ← I_global;                                   // First interaction
    return;
else
    pending ← YT.balanceOf(user) × (I_global − I_user[user]) / I_user[user];
    interest_owed[user] ← interest_owed[user] + pending;
    I_user[user] ← I_global;
end
```

4.3 Redemption Mechanisms

4.3.1 Pre-Expiry Redemption (PT + YT to SY)

Algorithm 5: Redeem PT and YT for SY (Pre-Expiry)

```
function RedeemPY(receiver, amount) → sy;
Input        : receiver: address, amount: u256
Output       : sy: amount of SY returned
Preconditions : block.timestamp < expiry;
receiver ≠ 0;
amount > 0;
PT.balanceOf(caller) ≥ amount;
YT.balanceOf(caller) ≥ amount;

// Update indices before balance changes
I_global ← max(I_global, current_exchange_rate);
UpdateUserInterest(caller);

// Burn equal PT and YT from caller
PT.balanceOf(caller) ← PT.balanceOf(caller) − amount;
PT.totalSupply ← PT.totalSupply − amount;
YT.balanceOf(caller) ← YT.balanceOf(caller) − amount;
YT.totalSupply ← YT.totalSupply − amount;

// Transfer SY from YT contract to receiver
SY.balanceOf(YT) ← SY.balanceOf(YT) − amount;
SY.balanceOf(receiver) ← SY.balanceOf(receiver) + amount;

emit RedeemPY(caller, receiver, amount);
return amount;

Post-invariant: PT.totalSupply = YT.totalSupply;

Reverts      : block.timestamp ≥ expiry: "use
               redeem_pt_after_expiry";
amount = 0: "zero amount";
receiver = 0: "zero address";
PT balance insufficient: "insufficient PT";
YT balance insufficient: "insufficient YT";
```

4.3.2 Post-Expiry Redemption (PT to SY)

Algorithm 6: Redeem PT for SY (Post-Expiry)

function REDEEMPTAFTEREXPIRY($receiver, amount$) $\rightarrow sy$;

Input : $receiver$: address, $amount$: u256

Output : sy: amount of SY returned (guaranteed 1:1)

Preconditions : `block.timestamp` $\geq expiry$;

$receiver \neq 0$;

$amount > 0$;

`PT.balanceOf`($caller$) $\geq amount$;

```
// No index update needed post-expiry
```

`PT.balanceOf`($caller$) $\leftarrow$ `PT.balanceOf`($caller$) $- amount$;

`PT.totalSupply` $\leftarrow$ `PT.totalSupply` $- amount$;

`SY.balanceOf`(YT) $\leftarrow$ `SY.balanceOf`(YT) $- amount$;

`SY.balanceOf`($receiver$) $\leftarrow$ `SY.balanceOf`($receiver$) $+ amount$;

emit REDEEMPTAFTEREXPIRY($caller, receiver, amount$);

return $amount$;

Reverts : `block.timestamp` $< expiry$: "not expired";

$amount = 0$: "zero amount";

$receiver = 0$: "zero address";

PT balance insufficient: "insufficient PT";

4.3.3 YT Interest Claim

Algorithm 7: Claim YT Interest

```
function ClaimInterest(receiver) → interest;
Input        : receiver: address
Output       : interest: amount of SY claimed
Preconditions : receiver ≠ 0
I_global ← max(I_global, current_exchange_rate);
UpdateUserInterest(caller);
interest ← interest_owed[caller];
interest_owed[caller] ← 0;
SY.balanceOf(YT) ← SY.balanceOf(YT) − interest;
SY.balanceOf(receiver) ← SY.balanceOf(receiver) + interest;
emit ClaimInterest(caller, receiver, interest);
return interest;
Reverts      : receiver = 0: "zero address";
SY.balanceOf(YT) < interest: "insufficient SY reserves";
```

4.4 AMM Mechanism

The Market contract implements a logit-based AMM for PT/SY trading with time-decay characteristics that ensure PT price converges to 1 at maturity.

4.4.1 Implied Rate Storage

The market stores the last implied rate to maintain price continuity across trades.

Variable	Type	Units	Description
r_{last}	u256	WAD-scaled	Natural log of implied annualized rate

Update formula. After each trade, the implied rate is recalculated from the new pool state:

$$r_{new} = \ln(\text{exchangeRate}) \times \frac{Y}{\tau}$$

where $Y = 31{,}536{,}000$ (seconds per year) and τ is time to expiry in seconds. The value is clamped to $[0, r_{max}]$ where $r_{max} = 4.6 \times 10^{18}$.

4.4.2 Precompute Values

Algorithm 8: Precompute Market Values

1 function PRECOMPUTE($state, t_{now}$) $\rightarrow pre$;

Input : $state$: MarketState, t_{now}: u64 (current timestamp)

Output : pre: MarketPrecompute struct

Preconditions : $t_{now} < state.expiry$

2 $\tau \leftarrow state.expiry - t_{now}$;

3 $r_{scalar} \leftarrow \dfrac{state.scalar_root \times Y}{\tau}$;

4 $p \leftarrow \dfrac{state.S_{PT}}{state.S_{PT} + state.S_{SY}}$;

5 $\text{exchangeRate}_{target} \leftarrow \exp\left(\dfrac{r_{last} \times \tau}{Y}\right)$;

6 $r_{anchor} \leftarrow \text{exchangeRate}_{target} - \dfrac{\text{logit}(p)}{r_{scalar}}$;

7 $\phi_{adjusted} \leftarrow \exp\left(\dfrac{\ln(\phi_{root}) \times \tau}{Y}\right)$;

8 **return** $\{r_{scalar}, r_{anchor}, \phi_{adjusted}, \tau\}$;

Reverts : $\tau = 0$: "expired";

$p \notin [p_{min}, p_{max}]$: "invalid state";

4.4.3 Price Calculation

The exchange rate depends on the current pool proportion:

$$\text{exchangeRate} = \frac{\text{logit}(p)}{r_{scalar}} + r_{anchor}$$

where $p = \frac{S_{PT}}{S_{PT}+S_{SY}}$ is the proportion of PT in the pool.

As expiry approaches, $r_{scalar} \rightarrow \infty$, which flattens the curve and forces the exchange rate toward 1.

4.5 Swap Mechanics

All swap functions reject trades that would violate proportion bounds or occur after expiry.

4.5.1 Swap Exact PT for SY

Algorithm 9: Swap Exact PT for SY

function SwapExactPTForSY($receiver, pt_{in}, sy_{min}$) $\rightarrow sy_{out}$;

Input : $receiver$: address, pt_{in}: u256, sy_{min}: u256 (slippage bound)

Output : sy_{out}: amount of SY received

Preconditions : `block.timestamp` $< expiry$;

$receiver \neq 0$;

$pt_{in} > 0$;

`PT.allowance`($caller, Market$) $\geq pt_{in}$;

$pre \leftarrow$ Precompute($state$, `block.timestamp`);

`// Calculate new proportion after adding PT`

$S'_{PT} \leftarrow state.S_{PT} + pt_{in}$;

$p' \leftarrow \dfrac{S'_{PT}}{S'_{PT} + state.S_{SY}}$;

if $p' > p_{max}$ **then revert** "proportion bounds exceeded";

`// Calculate exchange rate and output`

$\text{exchangeRate} \leftarrow \dfrac{\text{logit}(p')}{pre.r_{scalar}} + pre.r_{anchor}$;

if $exchangeRate \leq 0$ **then revert** "invalid exchange rate";

$sy_{gross} \leftarrow \dfrac{pt_{in} \times \text{exchangeRate}}{W}$;

$fee \leftarrow \dfrac{sy_{gross} \times (W - pre.\phi_{adjusted})}{W}$;

$sy_{out} \leftarrow sy_{gross} - fee$;

if $sy_{out} < sy_{min}$ **then revert** "slippage exceeded";

if $state.S_{SY} < sy_{out}$ **then revert** "insufficient reserves";

`// State changes`

$state.S_{PT} \leftarrow state.S_{PT} + pt_{in}$;

$state.S_{SY} \leftarrow state.S_{SY} - sy_{out}$;

$r_{last} \leftarrow \text{clamp}\left(\ln(\text{exchangeRate}) \times \dfrac{Y}{\tau}, 0, r_{max}\right)$;

`// Token transfers`

`PT.transferFrom`($caller, Market, pt_{in}$);

`SY.transfer`($receiver, sy_{out}$);

emit Swap($caller, receiver, pt_{in}, sy_{out}$);

return sy_{out};

4.5.2 Swap Exact SY for PT

Algorithm 10: Swap Exact SY for PT

function SwapExactSYForPT($receiver, sy_{in}, pt_{min}$) $\rightarrow pt_{out}$;

Input : $receiver$: address, sy_{in}: u256, pt_{min}: u256 (slippage bound)

Output : pt_{out}: amount of PT received

Preconditions : `block.timestamp` $< expiry$;

$receiver \neq 0$;

$sy_{in} > 0$;

`SY.allowance`($caller, Market$) $\geq sy_{in}$;

$pre \leftarrow$ Precompute($state$, `block.timestamp`);

`// Apply fee to input`

$fee \leftarrow \dfrac{sy_{in} \times (W - pre.\phi_{adjusted})}{W}$;

$sy_{net} \leftarrow sy_{in} - fee$;

$S'_{SY} \leftarrow state.S_{SY} + sy_{net}$;

`// Solve for` pt_{out} `via bisection`

$lo \leftarrow 0$;

$hi \leftarrow state.S_{PT} - L_{min}$;

for $i \leftarrow 1$ **to** 64 **do**

  $mid \leftarrow (lo + hi)/2$;

  $p' \leftarrow \dfrac{state.S_{PT} - mid}{state.S_{PT} - mid + S'_{SY}}$;

  $\text{rate} \leftarrow \dfrac{\text{logit}(p')}{pre.r_{scalar}} + pre.r_{anchor}$;

  $\text{implied} \leftarrow \dfrac{sy_{net} \times W}{\text{rate}}$;

  if $implied > mid$ **then**

    $lo \leftarrow mid$

  else

    $hi \leftarrow mid$

  end

  if $|hi - lo| \leq 1$ **then break**;

end

$pt_{out} \leftarrow lo$;

$p' \leftarrow \dfrac{state.S_{PT} - pt_{out}}{state.S_{PT} - pt_{out} + S'_{SY}}$;

if $p' < p_{min}$ **then revert** "proportion bounds exceeded";

if $pt_{out} < pt_{min}$ **then revert** "slippage exceeded";

`// State changes and transfers`

$state.S_{SY} \leftarrow state.S_{SY} + sy_{net}$;

$state.S_{PT} \leftarrow state.S_{PT} - pt_{out}$;

Update r_{last} as in SwapExactPTForSY;

`SY.transferFrom`($caller, Market, sy_{in}$);

Bisection rationale. The SY-to-PT direction requires solving for pt_{out} implicitly because the exchange rate depends on the post-trade proportion. Bisection is monotonic, bounded, and guaranteed to converge within 64 iterations for WAD-scaled values.

4.5.3 Inverse Swaps

Algorithm 11: Swap SY for Exact PT

```
function SwapSYForExactPT(receiver, pt_out, sy_max) → sy_spent;
Inverse of SwapExactSYForPT: given pt_out, solve for sy_in.;
Reverts : sy_spent > sy_max: "slippage exceeded"
```

Algorithm 12: Swap PT for Exact SY

```
function SwapPTForExactSY(receiver, sy_out, pt_max) → pt_spent;
Inverse of SwapExactPTForSY: given sy_out, solve for pt_in.;
Reverts : pt_spent > pt_max: "slippage exceeded"
```

4.6 Liquidity Provision

4.6.1 Add Liquidity (Mint LP)

Algorithm 13: Add Liquidity

1 function MINT($receiver, sy_{desired}, pt_{desired}$) $\rightarrow (sy, pt, lp)$;

Input : $receiver$: address, $sy_{desired}$: u256, $pt_{desired}$: u256

Output : (sy, pt, lp): amounts used and LP tokens minted

Preconditions : `block.timestamp` $< expiry$;

$receiver \neq 0$;

$sy_{desired} > 0 \wedge pt_{desired} > 0$;

2 **if** $L_{total} = 0$ **then**

`// Initial liquidity`

3 $lp \leftarrow \sqrt{sy_{desired} \times pt_{desired}} - L_{min}$;

4 $sy \leftarrow sy_{desired}$;

5 $pt \leftarrow pt_{desired}$;

`// Lock minimum to dead address`

6 MINTLP(`DEAD_ADDRESS`, L_{min});

7 **else**

`// Proportional liquidity`

8 $r_{sy} \leftarrow \dfrac{sy_{desired} \times W}{S_{SY}}$;

9 $r_{pt} \leftarrow \dfrac{pt_{desired} \times W}{S_{PT}}$;

10 $r \leftarrow \min(r_{sy}, r_{pt})$;

11 $sy \leftarrow \dfrac{r \times S_{SY}}{W}$;

12 $pt \leftarrow \dfrac{r \times S_{PT}}{W}$;

13 $lp \leftarrow \dfrac{r \times L_{total}}{W}$;

14 **end**

`// Verify proportion bounds after adding`

15 $p' \leftarrow \dfrac{S_{PT} + pt}{S_{PT} + pt + S_{SY} + sy}$;

16 **if** $p' \notin [p_{min}, p_{max}]$ **then revert** "proportion bounds";

17 $S_{SY} \leftarrow S_{SY} + sy$;

18 $S_{PT} \leftarrow S_{PT} + pt$;

19 $L_{total} \leftarrow L_{total} + lp$;

20 **emit** MINT($caller, receiver, sy, pt, lp$);

21 **return** (sy, pt, lp);

Reverts : expired, zero address, zero amounts, $lp = 0$, proportion bounds;

Constants. $L_{min} = 1000$ (locked to dead address on first mint). `DEAD_ADDRESS` = `0x...001`.

4.6.2 Remove Liquidity (Burn LP)

Algorithm 14: Remove Liquidity

1 function BURN($receiver, lp$) $\rightarrow (sy_{out}, pt_{out})$;
Input : $receiver$: address, lp: u256
Output : (sy_{out}, pt_{out}): amounts received
Preconditions : $receiver \neq 0$;
$lp > 0$;
`LP.balanceOf`($caller$) $\geq lp$;
$L_{total} - lp \geq L_{min}$;
2 $r \leftarrow \dfrac{lp \times W}{L_{total}}$;
3 $sy_{out} \leftarrow \dfrac{r \times S_{SY}}{W}$;
4 $pt_{out} \leftarrow \dfrac{r \times S_{PT}}{W}$;
5 $L_{total} \leftarrow L_{total} - lp$;
6 $S_{SY} \leftarrow S_{SY} - sy_{out}$;
7 $S_{PT} \leftarrow S_{PT} - pt_{out}$;
8 `LP.burn`($caller, lp$);
9 `SY.transfer`($receiver, sy_{out}$);
10 `PT.transfer`($receiver, pt_{out}$);
11 **emit** BURN($caller, receiver, sy_{out}, pt_{out}, lp$);
12 **return** (sy_{out}, pt_{out});
13 **Note:** Burning is allowed after expiry. LPs can always exit.;

4.7 Key Invariants

The protocol maintains two categories of invariants: safety invariants enforced by code, and economic invariants maintained by market incentives.

4.7.1 Safety Invariants (Enforced by Code)

These invariants are checked programmatically and violations cause transaction reversion.

Invariant	Expression	Enforcement
Supply equality	PT.totalSupply = YT.totalSupply	Atomic mint/burn in YT contract
Index watermark	$I_{global}^{t+1} \geq I_{global}^{t}$	max() in update function
Reserve positivity	$S_{SY} > 0 \wedge S_{PT} > 0$	L_{min} lock
Proportion bounds	$p_{min} \leq p \leq p_{max}$	Trade rejection
Rate bounds	$0 \leq r \leq r_{max}$	Clamping before storage
PT/YT mint authority	Only YT contract mints/burns	Access control check

4.7.2 Economic Invariants (Maintained by Arbitrage)

These invariants are expected to hold due to economic incentives but are not enforced on-chain.

Invariant	Expression	Mechanism
Conservation	$P_{PT} + P_{YT} \approx 1$ (in SY terms)	Arbitrage between mint/redeem and market
PT price convergence	$P_{PT} \to 1$ as $\tau \to 0$	AMM curve design + arbitrage
Implied rate consistency	Market rate $\approx$ fair rate	Arbitrage against external markets

Violation consequences.

- Supply equality violation: Accounting mismatch; claims exceed or fall short of reserves.
- Conservation deviation: Temporary mispricing; corrected by arbitrage.
- Index watermark violation: Loss of accrued yield for YT holders.

4.8 Failure Modes and Recovery

Each failure mode is documented with detection mechanism, on-chain mitigation, and residual risk.

4.8.1 Oracle Failure

Aspect	Description
Detection	Oracle returns stale data (age > MAX_ORACLE_AGE, default 1 hour)
On-chain response	Watermark index used; new yield not distributed until recovery
Trigger	Permissionless (watermark logic is automatic)
Residual risk	Yield generated during outage may not accrue to YT holders

4.8.2 Underlying Depeg

Scenario	Behavior
Temporary depeg	Index stays at watermark; no new yield distributed. Auto-recovery when rate exceeds watermark.
Permanent depeg	YT stops accruing yield indefinitely. PT redemption unaffected if SY remains solvent.
Complete failure	SY loses value. Users should exit via redemption. No protocol-level recovery.

4.8.3 Pool Imbalance

Aspect	Description
Detection	Proportion approaches p_{min} or p_{max}
On-chain response	Trades that exceed bounds are rejected
Self-correction	Extreme proportions create large arbitrage opportunities
Residual risk	Temporarily reduced liquidity in one direction

4.8.4 Flash Loan Attacks

Attack Vector	Mitigation
Price manipulation	Logit curve provides increasing resistance at proportion extremes
Sandwich attacks	User-specified slippage protection (sy_{min} / pt_{max})
Oracle manipulation	Pragma TWAP with configurable window (default: 5 minutes)

4.8.5 Expired Market Operations

Operation	Before Expiry	After Expiry
Swap	Allowed	Rejected
Mint LP	Allowed	Rejected
Burn LP	Allowed	Allowed
Redeem PT + YT	Allowed	Use RedeemPTAfterExpiry
Redeem PT only	Rejected	Allowed (1:1)
Claim Interest	Allowed	Allowed

5 Economic Analysis & Incentives

This section analyzes the economic properties of the protocol, including actor incentives, equilibrium conditions, and behavior under stress.

Scope and assumptions. All results in this section are conditional on the external assumptions stated in Chapter 3: oracle liveness, underlying asset solvency, and bounded transaction costs. Claims about equilibrium behavior assume competitive arbitrage and sufficient market liquidity.

5.1 Actors and Incentives

5.1.1 PT Holders (Fixed-Rate Exposure)

Decision rule. A rational actor buys PT when the market-implied fixed rate exceeds their required return:

$$r_{\text{implied}} \geq r_{\text{alt}} + \pi$$

where r_{alt} is the opportunity cost of capital and π is a risk premium covering contract risk, oracle risk, and underlying impairment risk.

Implied rate. Let Δ be time-to-expiry in years and P_{PT} the PT price in SY units. The implied annualized return is:

$$r_{\text{implied}} = P_{PT}^{-1/\Delta} - 1$$

For example, if $P_{PT} = 0.95$ and $\Delta = 0.5$ years, the implied APY is $(0.95)^{-2} - 1 \approx 10.8\%$.

Risks.

- **Smart contract risk**: Low probability, high severity. Mitigated by audits and formal verification.
- **Underlying impairment**: PT redemption is deterministic in SY units, but SY value depends on underlying solvency.

- **Opportunity cost**: Capital is locked; holder cannot chase higher yields elsewhere.
- **Early exit**: Must sell on AMM at prevailing price, subject to slippage and mark-to-market loss.

Implications. PT provides deterministic redemption in SY units at expiry, conditional on SY solvency. Prior to expiry, PT is mark-to-market and may trade at a discount or premium depending on rate expectations and liquidity.

5.1.2 YT Holders (Yield Speculation)

Decision rule. A rational actor buys YT when expected yield exceeds the YT price:

$$\mathbb{E}[g(\Delta)] \geq P_{YT}$$

where $g(\Delta)$ is the expected fractional yield growth of the underlying over time-to-expiry Δ. Equivalently, if y is the expected APY:

$$1 - e^{-y\Delta} \geq P_{YT}$$

This directly ties to the conservation identity $P_{PT} + P_{YT} = 1$.

Leverage effect. YT provides leveraged exposure to yield. If $P_{YT} = 0.05$ (5% of SY value) and the underlying yields 10% APY over a 6-month period:

- SY holder earns: $\approx 5\%$ on 100% capital
- YT holder earns: $\approx 5\%$ yield on 5% capital = $\approx 100\%$ effective return on YT investment

Risks.

- **Time decay**: YT value converges to zero as expiry approaches, regardless of yield.
- **Yield drops**: Reduced interest collection; YT loses value rapidly.
- **Complete depeg**: If underlying stops generating yield, YT becomes worthless.

Implications. YT is a high-risk, high-reward instrument suitable for actors with strong convictions about future yield. Longer maturities provide more leverage but greater time decay exposure.

5.1.3 Liquidity Providers

Decision rule. LP provision is attractive when expected return exceeds opportunity cost:

$$\mathbb{E}[\text{Fee APY}] + r_{SY} \cdot w_{SY} - \mathbb{E}[\text{IL}] \geq r_{\text{alt}}$$

where r_{SY} is the SY yield rate, w_{SY} is the SY weight in the pool, and IL is impermanent loss.

Impermanent loss dynamics. Unlike traditional AMMs where IL is symmetric, PT/SY pools exhibit asymmetric and time-dependent IL:

Rate Movement	PT Price	LP Impact
Rates increase	PT decreases	IL: LP holds excess PT
Rates decrease	PT increases	IL: LP holds excess SY
Near expiry	PT $\to$ 1	IL minimizes as prices converge

Time-to-expiry effect. As expiry approaches, the PT price becomes less sensitive to rate changes (via the rate scalar $r_{scalar} \to \infty$), reducing volatility-driven IL. However, this also typically reduces trading volume and fee opportunities. LPs face a tradeoff between IL protection and fee income as maturity nears.

Risks.

- **Directional IL**: Unlike xy=k pools, rate movements cause directional exposure shifts.
- **Rate volatility**: High volatility increases IL without proportional fee compensation.
- **Low volume periods**: Fee income may not offset capital opportunity cost.

Implications. LP returns are path-dependent and sensitive to rate volatility. Optimal entry is during stable rate environments; exit timing around volatility events is critical.

5.1.4 Arbitrageurs

Conservation bounds. The conservation identity creates soft bounds on prices:

$$P_{PT} + P_{YT} = 1 \pm \epsilon$$

where ϵ is bounded by transaction costs, slippage, and execution latency. In liquid markets with low gas costs, $\epsilon < 0.5\%$ is typical.

Arbitrage mechanisms.

Condition	Action	Effect
$P_{PT} + P_{YT} > 1 + \epsilon$	Mint PT+YT, sell both	Prices decrease
$P_{PT} + P_{YT} < 1 - \epsilon$	Buy PT+YT, redeem for SY	Prices increase
Horizon yield > external	Sell PT (or buy YT)	Implied yield decreases
Horizon yield < external	Buy PT (or sell YT)	Implied yield increases

Implications. Arbitrage activity maintains price consistency with external rate expectations and enforces the conservation identity within transaction cost bounds. Arbitrage profitability decreases as markets become more efficient.

5.2 Equilibrium Conditions

5.2.1 Price Equilibrium

At equilibrium, the implied yield reflects market consensus on future rates:

$$r_{\text{implied}} = \mathbb{E}[r_{\text{actual}}] + \pi$$

where π is a risk premium for contract and oracle risk. Deviations from this condition create arbitrage opportunities that are corrected by rational traders.

5.2.2 Liquidity Equilibrium

LP provision reaches equilibrium when marginal return equals opportunity cost:

$$\text{Fee APY} + r_{SY} \cdot w_{SY} - \mathbb{E}[\text{IL}] = r_{\text{alt}}$$

Factors affecting LP equilibrium depth:

- Higher trading volume ⇒ more fees ⇒ increased LP supply
- Higher rate volatility ⇒ more IL ⇒ decreased LP supply
- Higher underlying yield ⇒ more attractive SY exposure ⇒ increased LP supply

5.3 Stress Scenarios

5.3.1 Yield Collapse

Scenario. Underlying yield drops from 10% to 2% APY.

Actor	Immediate Impact	Final Outcome
PT Holder	Mark-to-market decline	Converges to 1 SY at maturity
YT Holder	YT price drops 80%+	Collects only 2% yield
LP	IL + reduced fee volume	Depends on exit timing

Key observation. PT holders who hold to maturity are unaffected in SY terms; the "loss" is opportunity cost relative to new higher-yielding alternatives.

5.3.2 Yield Spike

Scenario. Underlying yield spikes from 10% to 25% APY.

Actor	Immediate Impact	Final Outcome
PT Holder	Opportunity cost (missed yield)	Still receives 1 SY at maturity
YT Holder	Large windfall	Collects 25% yield; YT appreciates
LP	Directional IL	Higher volume partially offsets

5.3.3 Mass Redemption

Scenario. 80% of PT+YT holders redeem simultaneously.

Key property. Unlike lending protocols, the protocol has no maturity mismatch or run-induced insolvency risk, assuming SY is fully collateralized by the underlying. Each PT+YT pair is backed by exactly 1 SY held in the YT contract; redemption is always possible.

Constraints.

- Redemption throughput is limited by on-chain block capacity.
- AMM exits are subject to price impact and slippage.
- Solvency does not depend on borrower repayment (unlike lending protocols).

5.3.4 Underlying Depeg

Scenario. Underlying token loses 30% of external value.

Asset	Impact
SY	Worth 0.7× previous external value
PT	Redeems for 1 SY (now worth 0.7×)
YT	Yield continues at depressed rate
LP	Proportional loss on both reserve assets

Key distinction. Depeg affects the value of SY in external terms (e.g., USD), not the internal accounting identity PT $\rightarrow$ 1 SY. The protocol's internal guarantees remain intact; external value loss is passed through to all token holders.

Protocol response.

- Watermark index prevents distributing negative yield.
- All operations continue functioning normally.
- Users can exit at prevailing market prices.

5.4 Mechanism Properties

5.4.1 Equilibrium Characterization

The system admits a candidate equilibrium where no actor can improve their expected payoff by unilaterally deviating:

Actor	**Equilibrium Condition**
PT Buyer	Implied yield = required return for risk
YT Buyer	Expected yield $\geq$ YT price + speculation premium
LP	Fee income = IL + opportunity cost
Arbitrageur	Spread $\leq$ execution costs

Under competitive arbitrage and sufficient liquidity, deviations from equilibrium prices are mean-reverting.

5.4.2 Incentive Properties

Incentive compatibility. Honest behavior (accurate price discovery through trading) is locally incentive-compatible under competitive arbitrage and bounded manipulation costs. Price manipulation is costly (requires capital) and temporary (arbitrage corrects).

Individual rationality. Participation is voluntary. Expected utility is non-negative for informed participants who correctly assess risks and opportunities.

Budget balance. The protocol is self-sustaining: fees cover operational costs (gas for oracle updates, etc.). No external subsidies are required for equilibrium operation.

5.5 Fee Economics

5.5.1 Fee Retention Model

Fees are retained in pool reserves rather than extracted:

- Swap fees increase reserve balances (SY or PT depending on trade direction).

- LP token value appreciates proportionally to accumulated fees.
- Fee value is realized when LPs withdraw liquidity.

Terminology note. The swap mechanism uses a `fee_multiplier` $\in [0, 1]$ where:

$$\text{fee} = \text{gross_amount} \times (1 - \text{fee_multiplier})$$

A fee multiplier of 0.999 corresponds to a 0.1% fee rate.

5.5.2 Optimal Fee Rate

The optimal fee rate balances trader demand and LP incentives:

- **Too low**: Insufficient LP incentive ⇒ thin liquidity ⇒ high slippage.
- **Too high**: Traders use alternatives ⇒ low volume ⇒ reduced LP returns.

Market forces determine the effective optimum:

- Competition between markets for the same underlying sets fee ceiling.
- Arbitrage activity requires fees below arbitrage profit margins.
- LP entry/exit adjusts liquidity depth to equilibrium levels.

5.6 Value Accrual

5.6.1 Value Flow

Value Source	Recipient
Underlying yield	YT holders (until expiry)
PT discount capture	PT holders (at maturity)
Swap fees	LP holders
Arbitrage spreads	Arbitrageurs

Zero-sum framing. Discount capture (PT) and yield capture (YT) are two sides of the same decomposition: PT holders forgo yield in exchange for certainty; YT holders accept uncertainty for leveraged yield exposure. Fees and arbitrage spreads are paid by traders through execution price.

5.6.2 Protocol Value Capture

Current design. Zero protocol fee. All value flows to users (PT/YT/LP holders and arbitrageurs).

Future consideration. A governance-controlled protocol fee on swaps could direct value to a treasury or token holders. This would be implemented as a fraction of swap fees retained by the protocol rather than accruing to LPs.

6 Risk Analysis & Threat Model

This section enumerates the risks associated with the protocol, the controls in place, and residual risks that users must accept. Each risk category follows a consistent schema: threat, impact, preconditions, controls, and residual risk.

Threat model assumptions.

- The Starknet network operates correctly (liveness, validity proofs, L1 finality).
- The Pragma oracle infrastructure is not fundamentally compromised.
- Cryptographic primitives (hashing, signatures) remain secure.
- Users can monitor on-chain events and act within timelock windows.

6.1 Smart Contract Risk

6.1.1 Code Vulnerabilities

Threat	Controls	Status	Residual
Reentrancy	Checks-effects-interactions; explicit reentrancy guards; minimize external calls in state-mutating paths	Implemented	Low
Arithmetic overflow	Widened-precision math; explicit overflow checks; fuzz testing over boundary values	Implemented	Low
Access control bypass	Role-based access control (RBAC); explicit permission checks; separation of duties	Implemented	Low
Logic errors	Unit tests; integration tests; invariant fuzzing; external audit	Partial	Medium
Front-running / MEV	User-defined slippage bounds; min_out / max_in parameters	Implemented	Medium

Note on Cairo. Starknet's execution model differs from EVM but does not eliminate reentrancy-like patterns through external calls and callbacks. Explicit guards and patterns are required.

6.1.2 Dependency Risk

Dependency	Controls	Residual Risk
OpenZeppelin Cairo	Pinned versions; monitor security advisories	Upstream vulnerabil
cairo-fp (fixed-point math)	Extensive precision testing; boundary validation	Math library bugs
Starknet protocol	Accept as infrastructure dependency	Protocol-level bugs

6.2 Upgrade Risk

All core contracts are upgradeable using OpenZeppelin's Upgradeable component (class hash replacement). This section specifies the upgrade authority, controls, and user protections.

6.2.1 Upgrade Impact by Contract

Contract	Upgrade Can Modify	Impact
SY	Deposit/redeem logic; share accounting	Critical
PT	Redemption logic; mint authority	Critical
YT	Yield distribution; interest accounting	Critical
Market	AMM curve; swap logic; fee calculation	Critical
Router	Routing logic; could add hidden fees	High
Factory / MarketFactory	Implementation templates for new deployments	High

6.2.2 Upgrade Policy

Control	Status	Parameter	Residual
Multisig ownership	Planned	Target: 3-of-5 signers	Collusion risk
Timelock delay	Planned	Target: 48 hours minimum	Emergency bypass
Upgrade events	Implemented	`Upgraded(class_hash)` emitted	Requires monitoring
Code verification	Required	All class hashes verified on-chain	None
Pre-deployment audit	Policy	Audited before mainnet deployment	Audit scope limits

Emergency upgrade policy.

- Emergency upgrades (bypassing timelock) are permitted only for active exploits.
- Emergency actions require documented justification published within 24 hours.
- Post-emergency review and standard re-deployment within 7 days.

6.2.3 User Protections During Upgrades

Guarantee	Description
Exit window	Timelock provides users N hours to exit before upgrade execution.
Always-available operations	Even during pause: `redeem_pt_after_expiry`, `burn` (LP withdrawal), `claim_interest`.
Pausable operations	Swaps and new mints may be paused; existing positions remain accessible.
Upgrade transparency	Upgrade proposals include class hash diff and audit link (policy).

6.3 Oracle Risk

The protocol depends on Pragma oracle for yield index data used in YT interest accounting.

6.3.1 Oracle Parameters

Parameter	Value
TWAP window	5 minutes (configurable per SY)
Staleness threshold	3600 seconds (1 hour)
Data source	Pragma aggregated feed

6.3.2 Oracle Failure Behavior

Condition	Protocol Response	Rationale
Stale data (age > threshold)	Use watermark index; no new yield distributed	Protects against stale/manipulated data
Oracle unavailable	Operations using index continue with last value	Prevents liveness failure
Rate below watermark	Watermark unchanged; yield accounting frozen	Prevents negative yield distribution

Operations affected by oracle state.

- `mint_py`, `redeem_py`, `claim_interest`: Use current or watermark index.
- `redeem_pt_after_expiry`: Does not depend on oracle (1:1 redemption).
- AMM swaps: Do not use oracle; pricing is internal to pool state.

6.3.3 Exchange Rate Manipulation

Threat. Attacker manipulates oracle to extract value from the protocol.

Attack surfaces. The oracle index affects only the YT interest accounting system:

Subsystem	Oracle Dependency	Extraction Vector
Interest accounting	I_{global} updated from oracle	Inflated index ⇒ excess `claim_interest` payout
SY deposit/redeem	1:1 ratio (no oracle)	None
AMM pricing	Internal pool state only	None (oracle not used)

Attack scenario (index inflation).

1. Attacker manipulates oracle to report inflated exchange rate.
2. Attacker calls `claim_interest`; inflated I_{global} causes excess SY payout.
3. Oracle returns to normal; attacker has extracted SY from YT contract reserves.

Controls.

- TWAP (5-minute window) increases manipulation cost and duration required.
- Watermark prevents exploitation of rate *drops* (only increases can inflate index).
- Staleness check rejects data older than 1 hour.

Residual risk. Sustained oracle manipulation over the TWAP window can still inflate the index. Mitigation requires oracle-level security (Pragma's aggregation and data source diversity).

6.4 Market Risk

Market risks affect execution and returns but do not compromise protocol solvency. These are inherent to participation in yield markets.

6.4.1 Liquidity Risk

Scenario	Impact	User Mitigation
Thin liquidity	High slippage; inability to exit at fair price	Smaller orders; check price impact
LP concentration	Single LP can move market significantly	Monitor LP distribution
Extreme proportion	Trades rejected at bounds	Wait for arbitrage; use redemption path

Worst case. Inability to exit position at fair price before expiry. Protocol solvency is unaffected; this is execution risk only.

6.4.2 Impermanent Loss Risk

IL risk for LPs depends on rate volatility and time to expiry:

- **High volatility + long maturity**: Severe IL exposure; rate movements cause significant rebalancing.
- **Low volatility or short maturity**: Minimal IL; PT price convergence dominates.
- **Near expiry**: IL minimizes as $P_{PT} \to 1$ regardless of rate movements.

Key insight. IL risk decreases as expiry approaches because PT price becomes insensitive to rate changes ($r_{scalar} \to \infty$).

6.5 Underlying Asset Risk

Risks from the yield-bearing assets wrapped by SY tokens.

Risk	Protocol Behavior	User Impact
Depeg (value loss)	Watermark prevents negative yield accounting; redemption continues at 1:1 in SY	Principal impairment; users redeem into devalued asset
Yield cessation	YT stops accruing; PT converges toward 1 SY	YT holders receive no further yield; PT may reprice
Underlying protocol failure	SY may become worthless	Total loss possible
Regulatory freeze	Protocol cannot intervene	User bears external legal risk

Key distinction. The watermark protects yield accounting integrity (no negative yield distribution). It does *not* protect against principal impairment of the underlying asset. If the underlying loses value, all token holders (SY, PT, YT, LP) bear proportional losses.

6.6 Governance and Admin Risk

6.6.1 Admin Key Compromise

Threat. Attacker gains control of admin keys and executes malicious upgrades.

Impact. Complete protocol control; ability to drain funds via malicious upgrade.

Control	Status	Protection
Hardware wallet storage	Required	Reduces remote key theft
Multisig (3-of-5)	Planned	Requires multiple compromises or collusion
Timelock (48h)	Planned	Users can exit before malicious upgrade executes
Upgrade monitoring	Implemented	On-chain events enable automated alerts

6.6.2 Malicious Upgrade Detection

Attack	Detection Method	User Defense
Backdoor insertion	Code diff analysis; class hash verification	Exit during timelock window
Hidden fee extraction	Event monitoring; balance tracking	Review upgrade proposals
Logic manipulation	Automated contract comparison tools	Subscribe to upgrade alerts

6.6.3 Role Escalation

Risk	Control
Admin grants attacker roles	Multisig requirement; role changes emit events
Role accumulation	Separation of duties; distinct admin/pauser/operator roles
Pauser griefing	Pause is temporary; unpause requires same authority

6.7 Starknet Infrastructure Risk

External infrastructure risks outside protocol control.

Risk	Impact	User Response
Sequencer downtime	Transactions not processed	Wait for recovery; funds remain in L2 state
Sequencer censorship	Specific transactions blocked	Use alternative RPC; await escape mechanisms
Proof generation delay	Finality delayed on L1	Internal accounting unaffected; settlement certainty reduced
L1 congestion	High finalization costs	Accept as infrastructure cost
Protocol upgrade (Starknet)	Breaking changes possible	Monitor Starknet roadmap; protocol will adapt

Note on censorship. L1 force-inclusion mechanisms for Starknet are evolving. Until mature escape hatches exist, sequencer censorship remains an infrastructure-level risk that users accept.

6.8 User Operational Risk

User errors and operational security failures.

Risk	Description	Mitigation
Wrong address	Funds sent to incorrect recipient	Address validation; UI confirmations; verify full address
Address poisoning	Attacker creates similar-looking address	Use address book; verify complete address, not prefix
Wrong network/contract	Interaction with fake or test contract	Verify class hash; use official registry
Missed expiry	PT held past maturity	Redemption still works; no penalty
Slippage misconfiguration	Worse execution than expected	UI defaults; warnings on extreme values
Unlimited approvals	Excessive token permissions	Default to exact approvals; revoke after use
Approval phishing	Malicious approval requests	Wallet warnings; review approval targets
MEV sandwich	Execution between attacker trades	Tight slippage bounds; private mempools if available
Private key loss	Permanent fund loss	Hardware wallets; secure backup procedures

6.9 Threat Model Summary

6.9.1 Risk Priority Matrix

Risks are prioritized separately for systemic (protocol-wide) and individual (user-level) impact.

Risk	Likelihood	Systemic Impact	User Impact	Priority
Admin key compromise	Low	Critical	Critical	Critical
Smart contract bug	Low	Critical	Critical	Critical
Oracle manipulation	Medium	High	High	High
Underlying depeg	Low	High	High	High
Liquidity crisis	Medium	Low	Medium	Medium
Sequencer downtime	Low	Low	Low	Low
User operational error	High	None	High	Medium (user)

Interpretation.

- **Critical priority**: Requires maximum preventive investment and monitoring.
- **High priority**: Active controls required; residual risk accepted.
- **Medium priority**: Controls in place; ongoing monitoring.
- **Low priority**: Accepted as infrastructure or user-level risk.

6.10 Exclusions: What the Protocol Does NOT Protect Against

The following risks are explicitly outside the protocol's protection scope:

1. **Underlying asset impairment.** Principal loss from underlying asset failure is borne by users.
2. **Malicious or compromised governance.** Upgrade authority can modify contract logic; mitigated only by timelock and monitoring where implemented.

3. **Starknet network failures.** Protocol correctness depends on Starknet liveness and validity proofs.
4. **Oracle compromise.** Fundamental Pragma compromise could cause incorrect yield accounting; mitigated by TWAP and watermark but not eliminated.
5. **Regulatory actions.** Legal restrictions on assets or users cannot be prevented by the protocol.
6. **User operational errors.** Incorrect addresses, lost keys, and configuration mistakes are user responsibility.
7. **MEV and front-running.** Users can mitigate with slippage settings but cannot eliminate; this is inherent to public blockchains.

7 Governance & Upgrade Model

This section specifies the governance structure, upgrade capabilities, and admin powers. All protocol contracts are upgradeable; users must evaluate admin trust before depositing.

Authority model. All upgradeable contracts use OpenZeppelin's `OwnableComponent` and `UpgradeableComponent`. A single authority (the *owner*) holds `DEFAULT_ADMIN_ROLE` and can:

- Replace any contract's class hash via `upgrade(new_class_hash)`.
- Grant and revoke roles to other addresses.
- Execute contract-specific admin functions.

Target configuration: 3-of-5 multisig with 48-hour timelock (not yet deployed).

Upgrade semantics. Starknet contracts are upgraded by replacing the *class hash*—the on-chain identifier of the contract's bytecode. When `upgrade(new_class_hash)` executes:

1. The contract's class hash is atomically replaced.
2. Storage is preserved; the new implementation must be storage-compatible.
3. An `Upgraded(new_class_hash)` event is emitted.
4. All subsequent calls execute the new implementation immediately.

There is no proxy indirection; the contract address remains constant while its code changes.

7.1 Contract Classification

Horizon distinguishes between *internal* contracts (protocol-controlled) and *external dependencies* (third-party infrastructure).

7.1.1 Internal Contracts (Protocol-Controlled)

Contract	**Upgradeable**	raggedright**Scope of Upgrade Impact**	**Risk**
SY	Yes	raggedrightShare accounting, deposit/redeem logic	Critical
PT	Yes	raggedrightRedemption logic, mint authority validation	Critical
YT	Yes	raggedrightYield distribution, interest index accounting	Critical
Market	Yes	raggedrightAMM curve, swap execution, fee calculation	Critical
Factory	Yes	raggedrightTemplates for new SY/PT/YT deployments	High
MarketFactory	Yes	raggedrightTemplates for new Market deployments	High
Router	Yes	raggedrightRouting logic, could insert hidden fees	High
PragmaIndexOracle	Yes	raggedrightIndex source configuration, staleness thresholds	High

Risk interpretation.

Critical: Upgrade can directly drain or freeze user funds.

High: Upgrade can extract value or disrupt operations but cannot drain existing positions.

7.1.2 External Dependencies (Third-Party)

Dependency	**Trust Requirement**	**Failure Mode**
Starknet sequencer	Liveness, censorship resistance	Transactions not processed
Pragma oracle	Data integrity, freshness	Incorrect yield accounting
Underlying protocols (e.g., Nostra)	Solvency, correct yield accrual	SY value impairment

Dependency scope. The protocol cannot upgrade or control external dependencies. Users accept these as infrastructure-level trust assumptions. Failures in external dependencies can cause losses even if protocol code is correct.

7.2 Admin Powers

All admin functions require `DEFAULT_ADMIN_ROLE`. This section enumerates the exact on-chain powers.

7.2.1 Powers Available on All Upgradeable Contracts

Every contract with `OwnableComponent` and `UpgradeableComponent` exposes:

Function	raggedright**Effect**	**Risk**
`upgrade(class_hash)`	raggedrightReplace contract implementation atomically	Critical
`transfer_ownership(new_owner)`	raggedrightTransfer admin control to new address	Critical
`renounce_ownership()`	raggedrightPermanently remove admin control (irreversible)	N/A

7.2.2 Contract-Specific Admin Functions

Factory.

Function	raggedright**Effect**	**Risk**
`set_sy_class_hash`	raggedrightChange template for future SY deployments	High
`set_pt_class_hash`	raggedrightChange template for future PT deployments	Critical
`set_yt_class_hash`	raggedrightChange template for future YT deployments	Critical

Note. Class hash changes affect only *new* deployments. Existing SY/PT/YT instances retain their original implementation until individually upgraded.

MarketFactory.

Function	raggedright**Effect**	**Risk**
`set_market_class_hash`	raggedrightChange template for future Market deployments	Critical
`set_default_fee_rate`	raggedrightConfigure default swap fee for new markets	Medium
`set_default_scalar_root`	raggedrightConfigure default price sensitivity	Medium
`set_default_initial_anchor`	raggedrightConfigure default implied rate anchor	Medium

Router.

Function	raggedright**Effect**	**Risk**
`pause()`	raggedrightBlock all router operations	Medium
`unpause()`	raggedrightResume router operations	Low

PragmaIndexOracle.

Function	raggedright**Effect**	**Risk**
`set_staleness_threshold`	raggedrightConfigure maximum age for oracle data	Medium
`set_twap_duration`	raggedrightConfigure TWAP averaging window	Medium
`set_feed_id`	raggedrightChange Pragma feed identifier	High

Token Contracts (SY, PT, YT). Token contracts expose only `upgrade`. Minting/burning authority is enforced by code (YT contract controls PT/YT minting), not by admin-configurable roles.

Market. Market contracts expose only `upgrade`. AMM parameters (fee rate, scalar root, anchor) are set at deployment and cannot be modified post-deployment.

7.3 Role-Based Access Control

The protocol uses OpenZeppelin's `AccessControlComponent` with three roles:

Role	raggedright**Permissions**	**Holder**
`DEFAULT_ADMIN_ROLE`	raggedrightAll admin functions; grant/revoke other roles	Owner (multisig target)
`PAUSER_ROLE`	raggedright`pause()`, `unpause()` on Router	Owner (initially)
`OPERATOR_ROLE`	raggedrightReserved for future operational functions	Not assigned

Role hierarchy. `DEFAULT_ADMIN_ROLE` is the admin of all roles, including itself. Only addresses with `DEFAULT_ADMIN_ROLE` can grant or revoke any role.

7.4 Emergency Procedures

7.4.1 Emergency Powers

The following actions are available during emergencies:

Action	raggedright**Effect**
Pause Router	raggedrightBlocks new swaps, mints, and complex operations via Router
Emergency upgrade	raggedrightReplace contract implementation without timelock (if active exploit)

7.4.2 Pause Scope

The pause mechanism affects only the Router contract. Its purpose is to stop new protocol interactions while preserving user exit paths.

Operation	**During Pause**	raggedright**Rationale**
Router swaps	Blocked	raggedrightPrevent exploitation via complex operations
Router minting (PT/YT)	Blocked	raggedrightPrevent new positions during incident
Router liquidity add/remove	Blocked	raggedrightPrevent position changes
Direct SY deposit/redeem	Available	raggedrightUsers can exit to underlying
Direct PT transfer	Available	raggedrightUsers can move positions
Direct YT transfer	Available	raggedrightUsers can move positions
Post-expiry PT redemption	Available	raggedrightUsers can redeem matured positions
YT interest claim	Available	raggedrightUsers can claim accrued yield
Market direct swap	Available	raggedrightDirect contract calls bypass Router
LP burn (withdraw liquidity)	Available	raggedrightUsers can exit LP positions

User Safety Invariant During Emergencies
Even when the Router is paused, users can always:

1. Redeem SY for underlying assets (direct call to SY contract).
2. Redeem PT for SY after expiry (direct call to YT contract).
3. Claim accrued YT interest (direct call to YT contract).
4. Withdraw LP positions by burning LP tokens (direct call to Market).
5. Transfer all token types to other addresses.

No emergency action can prevent users from exiting existing positions.

7.4.3 Emergency Response Process

1. **Detection.** Exploit identified via monitoring, user report, or external disclosure.

2. **Triage.** Assess severity and funds at risk. Classify as Critical/High/Medium/Low.
3. **Containment.** If Critical or High: immediately pause Router.
4. **Remediation.** Develop and test fix. For Critical exploits, emergency upgrade without timelock.
5. **Disclosure.** Publish incident report within 24 hours of resolution.
6. **Post-mortem.** Conduct root cause analysis; implement preventive measures.

Timelock bypass justification. Emergency upgrades (bypassing any future timelock) are permitted only when:

- Active exploitation is occurring or imminent.
- Funds at risk exceed a material threshold.
- Waiting for timelock would result in greater user losses.

7.5 Upgrade Process

7.5.1 Standard Upgrade Steps

1. **Development.** Implement changes; ensure storage compatibility.
2. **Testing.** Full test suite passes; invariant fuzzing on modified paths.
3. **Audit.** External audit for significant changes (new features, security-relevant modifications).
4. **Proposal.** Publish upgrade proposal with:
   - New class hash
   - Code diff or audit report link
   - Rationale and risk assessment
5. **Timelock.** (When implemented) Submit upgrade transaction; wait for timelock period.
6. **Execution.** Multisig signers approve and execute upgrade.
7. **Verification.** Confirm new class hash matches proposal; verify contract behavior.

7.5.2 Storage Compatibility Requirements

Upgrades must preserve storage layout:

- Existing storage variables retain their slots.
- New variables are appended (not inserted).
- Variable types are not changed.
- Mappings and structs maintain compatible layouts.

Incompatible upgrades. If storage layout must change, a migration pattern is required: deploy new contract, migrate state, update references. This is a breaking change requiring extended user notice.

7.6 Governance Attack Surface

Attack Vector	Likelihood	Impact	raggedrightMitigations
Admin key compromise	Low	Critical (total loss)	raggedrightHardware wallet; multisig (planned); timelock (planned)
Malicious insider upgrade	Low	Critical (total loss)	raggedrightMultisig threshold; code review; audit requirement
Social engineering of signers	Medium	Critical	raggedrightDedicated signing devices; verification procedures
Compromised upgrade proposal	Low	Critical	raggedrightClass hash verification; multiple reviewer sign-off
Role escalation	Very Low	High	raggedrightRBAC structure; role change events; monitoring
Pause abuse (griefing)	Low	Medium (temporary)	raggedrightPAUSER_ROLE separation; unpause requires same authority

Attack surface reduction. The governance attack surface is proportional to:

- Number of addresses with admin roles (currently 1; target 5 multisig signers).
- Frequency of upgrades (minimize by shipping stable code).
- Complexity of upgrade process (reduce via automation and verification tooling).

7.7 Trust Assumptions

Users must evaluate what they trust before using the protocol.

7.7.1 Trusted (Cannot Be Verified)

- Admin keys are not compromised.
- Multisig signers do not collude maliciously.
- Upgrades are deployed with benevolent intent.
- Emergency actions are taken in good faith.
- Operational security of key holders is maintained.

7.7.2 Verifiable On-Chain

- All contract code (class hashes can be verified against source).
- All role grants and revocations (events emitted).
- All upgrades (events emitted with new class hash).
- Current pause state.
- All token balances and positions.
- AMM state and pool reserves.

7.7.3 Trustless (Given Current Code)

- Mathematical correctness of AMM pricing and yield accounting.
- Token conservation (no inflation without corresponding deposits).
- Access control enforcement (only authorized addresses can call admin functions).
- Expiry mechanics (PT redeems 1:1 after expiry, YT stops accruing).

Key insight. Upgradeability means "trustless" properties hold only for the *current* implementation. A malicious upgrade can violate any property. Users must trust admin or verify each upgrade.

7.8 Governance Roadmap

The following governance features are planned but not yet implemented:

Multisig (3-of-5). Require multiple independent signers for all admin actions. Prevents single-key compromise from causing total loss.

Timelock (48 hours). Delay between upgrade proposal and execution. Provides users time to review changes and exit if concerned.

Upgrade monitoring. Automated alerts when upgrade transactions are submitted. Enables community review during timelock window.

Verified class hash registry. On-chain registry of audited, approved class hashes. Upgrades restricted to pre-verified implementations.

Decentralization considerations. Full decentralization (governance token, on-chain voting) is not planned for initial deployment. Rationale:

- Governance tokens introduce new attack vectors (vote buying, governance attacks).
- Early-stage protocols benefit from rapid iteration capability.
- Decentralization is meaningful only with sufficient distribution and participation.

The protocol may consider decentralized governance once the codebase is stable and user base is established.

8 Related Work

This section positions Horizon relative to existing yield and fixed-rate protocols. The comparison focuses on mechanism design, capital efficiency, precision guarantees, and execution environment rather than superficial feature parity.

8.1 Pendle Finance (Design Lineage)

Horizon intentionally adopts Pendle's logit-based AMM and PT/YT conservation model. This is a deliberate design choice: Pendle's mechanics have been battle-tested across multiple market cycles, exhibit well-understood convergence behavior, and create predictable incentives for LPs and arbitrageurs. By preserving design parity, Horizon minimizes economic novelty risk while innovating at the execution and infrastructure layer.

8.1.1 Design Parity

The following elements are intentionally equivalent:

- **Logit-based exchange rate:** rate $= \frac{\ln(p/(1-p))}{r_{scalar}} + r_{anchor}$
- **Time-decaying rate scalar:** Sensitivity increases as expiry approaches.
- **Conservation invariant:** PT + YT = SY at all times.
- **Exponential fee decay:** Fees decrease toward expiry to encourage late-market trading.
- **Yield accounting:** Global index tracks cumulative yield; user snapshots determine claimable interest.

Rationale. Economic equivalence means Horizon inherits Pendle's proven market dynamics: arbitrage bounds implied rates, LPs face predictable IL profiles, and users can reason about positions using existing Pendle mental models. This reduces adoption friction and audit scope.

8.1.2 Execution Divergence

The following elements differ due to execution environment:

Aspect	Pendle (EVM)	Horizon (Starknet)
Math library	LogExpMath (Solidity)	cairo-fp 64.64 fixed-point
Precision	∼18 decimal digits	∼19 decimal digits (wider intermediates)
Overflow model	Manual checks; revert on overflow	Cairo bounds checking; felt252 range
Oracle integration	Chainlink + internal rate tracking	Pragma TWAP with configurable staleness
Upgrade model	Transparent proxy (EIP-1967)	Class hash replacement (native Starknet)
Gas cost per swap	High (L1) / Medium (L2)	Low (validity rollup)

Precision advantage. The logit function $\ln(p/(1-p))$ requires high-precision transcendental math. Solidity implementations use lookup tables and approximations; Cairo's 64.64 fixed-point library computes ln and exp with lower approximation error, reducing pricing drift at extreme proportions.

8.2 Element Finance (Historical Precedent)

Element Finance pioneered on-chain yield tokenization, demonstrating market demand for PT/YT separation. The protocol operated from 2021–2023 before discontinuing active development.

8.2.1 Element's Contribution

Element validated the core thesis: users want to trade yield separately from principal. The protocol successfully tokenized positions from Yearn vaults, Compound, and Aave, proving that yield tokenization is economically viable.

8.2.2 YieldSpace Limitations

Element used the YieldSpace AMM, a constant-power curve designed for yield trading:

$$x^{1-t} + y^{1-t} = k$$

where t represents time to maturity. This curve has known limitations:

- **Liquidity fragmentation:** Each expiry requires a separate pool; liquidity cannot be shared.
- **Capital inefficiency near maturity:** As $t \to 0$, the curve flattens but does not guarantee $P_{PT} \to 1$.
- **Unpredictable convergence:** Final PT price depends on pool composition, not mathematical constraint.

Lesson applied. Horizon's logit-based curve directly addresses these limitations. The rate scalar $r_{scalar} \to \infty$ as $t \to 0$ mathematically forces $P_{PT} \to 1$, eliminating convergence uncertainty. LPs near expiry face minimal IL because PT price becomes insensitive to trading.

8.3 Fixed-Rate Lending Protocols

Several protocols offer fixed-rate exposure through lending markets rather than yield tokenization. The mechanisms differ fundamentally.

Protocol	Mechanism	Risk Type
Notional Finance	Fixed-rate lending via borrow/lend matching	Counterparty risk (borrower default)
Yield Protocol	Zero-coupon tokens (fyTokens) via secondary trading	Counterparty risk (undercollateralization)
Aave (stable rate)	Variable-to-stable conversion via protocol reserve	Protocol solvency risk
Horizon	Yield tokenization of existing yield-bearing assets	Underlying asset risk only

Key Economic Distinction
Horizon does not create yield through leverage or rehypothecation. All returns derive from existing yield-bearing assets (e.g., staked ETH, lending deposits). This eliminates:

- Borrower default risk (no borrowers exist)
- Liquidation cascade risk (no collateral to liquidate)
- Utilization-dependent rate volatility (yield comes from underlying, not borrow demand)

PT holders receive exactly 1 SY at expiry regardless of market conditions—the only risk is impairment of the underlying asset itself.

Liquidity risk comparison. In lending protocols, liquidity depends on borrow demand and utilization ratios. In yield tokenization, liquidity depends on AMM depth. Both create execution risk, but the risk sources are independent: lending liquidity crises (high utilization) do not affect Horizon markets, and vice versa.

8.4 Starknet Execution Environment

Horizon's design requires specific execution environment properties. This section distinguishes between properties critical to the protocol's mechanics and incidental benefits.

8.4.1 Protocol-Critical Properties

The following Starknet characteristics are necessary for Horizon's design:

High-precision transcendental math. The logit function requires ln and exp with low approximation error. Cairo's 64.64 fixed-point library provides sufficient precision; EVM alternatives require lookup tables or iterative approximations that accumulate error.

Low-cost arbitrage. Implied rate accuracy depends on arbitrageurs correcting mispricing. If arbitrage is unprofitable due to gas costs, rates diverge from fair value. Starknet's low transaction costs keep arbitrage profitable even for small deviations.

Affordable oracle updates. Yield accounting requires frequent index updates. On Ethereum L1, updating yield indices on every relevant transaction would be prohibitively expensive. Starknet's cost structure makes per-transaction oracle reads economically viable.

Native STARK verification. Starknet's validity proofs ensure transaction correctness without trust assumptions beyond cryptographic soundness. This complements the protocol's trust model.

8.4.2 Incidental Benefits

The following properties improve user experience but are not strictly necessary:

- **Fast perceived finality:** L2 confirmation in seconds improves UX for active traders.
- **Lower barrier to entry:** Reduced gas costs make small positions economically viable.
- **Growing DeFi ecosystem:** Availability of yield-bearing assets (nstSTRK, sSTRK, wstETH) provides initial markets.

Alternative environments. Other validity rollups (e.g., zkSync, Polygon zkEVM) could theoretically support similar designs. Starknet was chosen for its mature Cairo tooling, native fixed-point libraries, and Pragma oracle integration. The protocol's design is not fundamentally Starknet-exclusive, but porting would require reimplementing the math libraries.

8.5 Design References and Prior Work

The following works inform Horizon's design:

Reference	Year	Design Contribution
Pendle Litepaper	2022	Logit-based AMM curve; PT/YT conservation model
YieldSpace (Yield Protocol)	2021	Constant-power curves for yield trading; time-decay mechanics
Uniswap v2 Whitepaper	2020	Constant product AMM; LP share accounting
Curve StableSwap	2019	Low-slippage trading for pegged assets; concentrated liquidity
Balancer Whitepaper	2020	Weighted pools; generalized invariant surfaces

Design lineage. Horizon's AMM inherits from Curve's insight that asset-specific invariants outperform general-purpose curves, combined with Pendle's application of this principle to yield markets. The time-decay mechanism originates from YieldSpace but is implemented via the logit transformation rather than constant-power exponents.

8.6 Positioning Summary

Horizon occupies a specific position in the yield protocol landscape:

Comparison	Horizon's Position
vs Pendle	Equivalent economics; different execution environment (Starknet vs EVM)
vs Element	Same tokenization model; superior AMM (logit vs constant-power)
vs Fixed-rate lending	Fully asset-backed; no counterparty risk; no leverage
vs Generic AMMs	Purpose-built for yield; deterministic expiry convergence

Value proposition. Horizon combines Pendle's proven yield tokenization mechanics with Starknet's execution environment to achieve higher arithmetic precision, lower trading friction, and native oracle integration. Compared to fixed-rate lending, Horizon is fully asset-backed and non-leveraged. Compared to earlier yield AMMs, it provides deterministic price convergence and improved capital efficiency near expiry.

The protocol does not claim novel economic mechanisms. Its contribution is bringing battle-tested yield tokenization to a new execution environment with properties—

low-cost arbitrage, high-precision math, affordable oracle updates—that enhance the existing design rather than replace it.

9 Conclusion

9.1 Conclusion

Horizon Protocol introduces a yield tokenization framework on Starknet, enabling users to separate principal and yield exposure for yield-bearing assets.

Key contributions.

- **Starknet-native yield tokenization.** A PT/YT model adapted to Starknet's execution and arithmetic model, providing the first yield tokenization infrastructure on the network.
- **Cairo-native numerical design.** High-precision fixed-point arithmetic via the cairo-fp 64.64 library, enabling accurate transcendental function computation for logit-based pricing.
- **Native oracle integration.** Pragma TWAP-based exchange rate tracking for yield-bearing assets, with configurable staleness thresholds and watermark protection.
- **Logit-based AMM implementation.** A complete implementation of the Pendle-style curve with deterministic price convergence at expiry.

9.2 Core Guarantees

The following guarantees define the protocol's economic correctness and distinguish it from lending-based fixed-rate systems.

Guarantee	Mechanism	Enforcement
PT redemption at par	PT $\rightarrow$ 1 SY after expiry	Contract logic
Token conservation	PT + YT = SY always holds	Arbitrage + contract
Deterministic convergence	$r_{scalar} \rightarrow \infty$ as $\tau \rightarrow 0$	AMM mathematics
Supply equality	PT.supply = YT.supply	Contract assertion
Index monotonicity	I_{global} only increases	Watermark logic

Together, these guarantees ensure that PT holders receive exactly 1 SY at expiry regardless of market conditions, YT holders receive all accrued yield, and LPs face predictable convergence dynamics.

9.3 Design Trade-offs

The following trade-offs were intentionally chosen to balance correctness, complexity, and deployability for a v1 protocol.

Trade-off	Choice	Rationale
Upgradeability vs immutability	Upgradeable	Enables bug fixes; requires admin trust
Curve complexity vs simplicity	Full logit curve	Capital efficiency justifies complexity
Gas cost vs precision	cairo-fp 64.64	Financial math requires high precision
Single-block vs multi-block execution	Single-transaction	Reduces state complexity; oracle TWAP mitigates manipulation

Note on execution model. All protocol operations complete within a single transaction. Multi-block execution patterns (e.g., commit-reveal) were not implemented. Oracle manipulation resistance relies on Pragma's TWAP mechanism rather than protocol-level multi-block designs.

9.4 Out-of-Scope Enhancements

The following enhancements are out of scope for this specification and are not commitments. They are listed to clarify possible future directions.

Enhancement	Purpose
Timelock for upgrades	Allow users to exit before upgrade execution
Multisig requirement	Reduce single-key compromise risk
Additional SY adapters	Support more yield-bearing assets
Cross-market routing	Optimize multi-hop trade execution
Governance token	Enable decentralized protocol control

These items may be addressed in future protocol versions but are explicitly excluded from this specification.

9.5 Specification Status

This document specifies Horizon Protocol v1.0.

Component	Status	Coverage
Token model (SY, PT, YT)	Complete	Mint, redeem, transfer, yield accounting
AMM mechanism	Complete	Logit curve, fee decay, liquidity operations
Redemption flows	Complete	Pre-expiry and post-expiry paths
Oracle integration	Complete	Pragma TWAP, staleness, watermark
Governance model	Complete	Upgradeability, RBAC, emergency procedures
Risk analysis	Complete	Threat model, attack surfaces, mitigations
Parameter specification	Complete	Bounds, defaults, configuration

This document constitutes the complete technical specification for Horizon Protocol v1.0. Any behavior not described herein is considered undefined.

A Mathematical Proofs

This appendix formalizes the key mathematical properties that underpin Horizon's economic guarantees.

A.1 PT Price Convergence

Theorem A.1. Assuming a bounded implied rate $r < \infty$, the PT price converges to its redemption value as expiry approaches:

$$\lim_{\tau \to 0} P_{PT} = 1.$$

Proof.

The PT price is defined as:

$$P_{PT} = e^{-r \cdot \tau_y}, \quad \text{where} \quad \tau_y = \frac{\tau}{Y}$$

and Y is the number of seconds per year (31,536,000).

As $\tau \to 0$, we have $\tau_y \to 0$. Since r is bounded by construction ($0 \leq r \leq r_{max}$), the exponent satisfies:

$$\lim_{\tau \to 0}(-r \cdot \tau_y) = 0.$$

By continuity of the exponential function:

$$\lim_{\tau \to 0} e^{-r \cdot \tau_y} = e^0 = 1.$$

Therefore, $P_{PT} \to 1$ as $\tau \to 0$. □

A.2 Conservation Invariant

Theorem A.2. For all times $t < T$ (before expiry), the following invariant holds:

$$1\,\text{PT} + 1\,\text{YT} \longleftrightarrow 1\,\text{SY}.$$

Proof.

Before expiry, the protocol exposes two mutually inverse state transitions:

- **Split:** Depositing x SY mints exactly x PT and x YT.
- **Recombine:** Burning x PT and x YT releases exactly x SY.

By construction:

$$\text{PT.supply} = \text{YT.supply}$$

and all outstanding PT and YT are fully backed by SY held in the YT contract:

$$\text{SY}_{\text{YT}} = \text{PT.supply} = \text{YT.supply}.$$

No operation allows PT or YT to be minted or burned independently. Therefore, for all valid states before expiry:

$$1\,\text{PT} + 1\,\text{YT} = 1\,\text{SY}.$$

□

A.3 No-Arbitrage Pricing Condition

Theorem A.3. In the absence of fees, slippage, and execution costs, any deviation from

$$P_{PT} + P_{YT} = 1$$

admits a risk-free arbitrage opportunity.

Assumptions.

- Zero swap fees
- Zero slippage (infinite liquidity)
- Zero gas/execution costs
- Instantaneous execution

Proof.

Case 1: $P_{PT} + P_{YT} > 1$

1. Purchase 1 SY at unit price.
2. Split into 1 PT and 1 YT.
3. Sell PT for P_{PT} and YT for P_{YT}.

Net profit:

$$P_{PT} + P_{YT} - 1 > 0.$$

Case 2: $P_{PT} + P_{YT} < 1$

1. Purchase 1 PT for P_{PT} and 1 YT for P_{YT}.
2. Recombine into 1 SY.
3. Sell SY at unit price.

Net profit:

$$1 - P_{PT} - P_{YT} > 0.$$

In both cases, arbitrageurs are incentivized to trade until the equality

$$P_{PT} + P_{YT} = 1$$

is restored, up to execution costs.

□

Remark. In practice, fees and slippage introduce a bounded arbitrage band around the equality. If total round-trip costs are ϵ, then the no-arbitrage condition relaxes to:

$$|P_{PT} + P_{YT} - 1| \leq \epsilon.$$

Within this band, arbitrage is unprofitable and deviations may persist.

B Numerical Examples

The following examples illustrate concrete protocol behavior under simplified assumptions. They are intended for intuition-building and do not model fees, slippage, or adversarial market conditions unless explicitly stated. All calculations use continuous compounding consistent with the protocol's pricing model.

B.1 PT Purchase Flow

Scenario. A user purchases PT to lock in an implied yield of 10% APY on 1000 SY, with 180 days to expiry.

Assumptions.

- No swap fees or slippage
- Sufficient AMM liquidity (price impact negligible)
- Continuous compounding, consistent with protocol pricing

Given:

- SY amount: 1000
- Implied yield: $r = 0.10$
- Time to expiry: $\tau_y = \frac{180}{365} \approx 0.493$ years

PT Price:

$$P_{PT} = e^{-r \cdot \tau_y} = e^{-0.10 \times 0.493} \approx 0.9520$$

PT Amount Received:

$$\text{PT} = \frac{1000}{0.9520} \approx 1050.4$$

At Maturity:

$$\text{SY received} = 1050.4 \times 1 = 1050.4$$

Profit:

$$\text{Profit} = 1050.4 - 1000 = 50.4 \text{ SY}$$

Implied APY Check:

$$\left(\frac{1050.4}{1000}\right)^{1/0.493} - 1 \approx 10.5\%$$

Note: The discrete APY calculation yields approximately 10.5% due to the difference between discrete and continuous compounding. Under continuous compounding, the return is exactly 10% annualized.

Interpretation. The PT holder sacrifices liquidity and yield upside in exchange for a fixed implied return that is independent of future yield fluctuations. The return is locked at purchase and realized at maturity.

B.2 LP Deposit and Withdrawal

Scenario. An LP deposits liquidity into a PT/SY market and later withdraws after fees accrue.

Assumptions.

- PT price remains constant during the period
- No external trades besides fee-generating swaps
- PT valued at 1 SY for accounting simplicity (appropriate near expiry)

Initial Pool State:

- Reserves: 1000 SY + 1000 PT
- Total LP supply: 1000

Deposit:

- LP deposits: 500 SY + 500 PT
- Deposit ratio: $\frac{500}{1000} = 0.5$
- LP minted: $0.5 \times 1000 = 500$

Post-Deposit Pool:

- Reserves: 1500 SY + 1500 PT
- Total LP supply: 1500

After Fee Accumulation:

- Fees earned by pool: 10 SY
- Pool reserves: 1510 SY + 1500 PT

Withdrawal (500 LP):

$$\text{Ownership share} = \frac{500}{1500} = 0.333$$

$$\text{SY out} = 0.333 \times 1510 \approx 503.3$$

$$\text{PT out} = 0.333 \times 1500 = 500$$

LP Profit (Fee Income):

$$\text{Fee share} = 503.3 - 500 = 3.3 \text{ SY}$$

The LP receives their pro-rata share of the 10 SY in fees ($\frac{500}{1500} \times 10 = 3.3$ SY).

Remark. This example isolates fee income by assuming constant PT price. In practice, PT price movements introduce impermanent loss. However, IL risk diminishes as expiry approaches because $P_{PT} \to 1$ regardless of rate movements.

B.3 YT Interest Claim

Scenario. A user holds YT while the underlying yield index increases.

Index Semantics. The global index I_{global} represents the cumulative exchange rate of the underlying yield-bearing asset. When the underlying earns yield, I_{global} increases. The user's snapshot I_{user} records the index value at the time of their last claim (or initial acquisition).

Given:

- YT balance: $B_{YT} = 100$
- User index (snapshot): $I_{user} = 1.00$

- Global index (current): $I_{global} = 1.05$

Interest Calculation:

$$\text{Interest} = B_{YT} \times \frac{I_{global} - I_{user}}{I_{user}} = 100 \times \frac{1.05 - 1.00}{1.00} = 5 \text{ SY}$$

Post-Claim State: After claiming, the user's index is updated:

$$I_{user} \leftarrow I_{global} = 1.05$$

Future claims will only capture yield accrued after this point.

Interpretation. The YT holder captures the full 5% increase in the underlying yield index (5 SY on 100 YT). PT holders receive no interim yield—their return is locked at the PT price discount. This separation allows users to speculate on yield rates or hedge yield exposure independently of principal.

B.4 Assumptions Summary

The examples above make the following simplifying assumptions:

Assumption	**Real-World Consideration**
Zero swap fees	Protocol charges fees; reduces effective return
Zero slippage	Large trades move price; reduces execution quality
Infinite liquidity	AMM has finite depth; large trades have price impact
PT price constant	PT price fluctuates with implied rate; creates IL for LPs
Instant execution	Block time and sequencer latency introduce timing risk
WAD rounding omitted	All amounts are scaled by 10^{18}; rounding affects dust

These assumptions are appropriate for understanding protocol mechanics but should not be used for precise financial projections.

C Contract Interface Specifications

This appendix specifies the external interfaces for all core protocol contracts. Each interface includes semantic invariants, parameter specifications, and authority constraints.

Terminology. Throughout this appendix, **PY** denotes a matched pair of PT and YT tokens (1 PY = 1 PT + 1 YT). All amounts are WAD-scaled (10^{18}) unless otherwise noted. Timestamps are Unix seconds.

C.1 Interface Summary

Interface	Responsibility	Trust Level	Key Invariant
SY	Yield abstraction layer	Medium	Fully backed by underlying
PT	Principal token claim	High	Supply equals YT supply
YT	Yield accounting and distribution	High	Index monotonically increases
Market	PT/SY price discovery	Medium	Proportion bounds enforced

C.2 SY Interface

Semantic invariants.

- SY is always fully backed by underlying assets held by the contract.
- `exchange_rate()` is monotonically non-decreasing (yield only accrues, never decreases).
- Total SY supply equals the underlying value divided by exchange rate.

```
#[starknet::interface]
pub trait ISY<TContractState> {
    // ERC20 standard functions (name, symbol, decimals,
```

```
    // total_supply, balance_of, transfer, approve, transfer_from)

    fn deposit(
        ref self: TContractState,
        receiver: ContractAddress,
        token_in: ContractAddress,
        amount_token_to_deposit: u256,
        min_shares_out: u256
    ) -> u256;

    fn redeem(
        ref self: TContractState,
        receiver: ContractAddress,
        amount_shares_to_redeem: u256,
        token_out: ContractAddress,
        min_token_out: u256,
        burn_from_internal_balance: bool
    ) -> u256;

    fn exchange_rate(self: @TContractState) -> u256;
    fn underlying(self: @TContractState) -> ContractAddress;
    fn yt(self: @TContractState) -> ContractAddress;
}
```

Parameter specification.

Parameter	Type	Description
`amount_token_to_deposit`	u256	Underlying amount (WAD-scaled)
`min_shares_out`	u256	Minimum SY to receive; reverts if not met
`amount_shares_to_redeem`	u256	SY amount to burn (WAD-scaled)
`token_in / token_out`	address	Underlying token address (multi-asset support)
`exchange_rate` return	u256	Current rate (WAD-scaled); $\geq 1 \times 10^{18}$

Notes.

- `token_in` / `token_out` enable SY implementations that support multiple underlying assets (e.g., wrapped vs unwrapped).

- `burn_from_internal_balance` allows gas-efficient redemptions when SY is already held by the contract.
- `deposit` reverts if `min_shares_out` is not met (slippage protection).

C.3 PT Interface

Semantic invariants.

- PT is non-inflationary: the only mint/burn authority is the YT contract.
- PT supply always equals YT supply.
- After expiry, each PT is redeemable for exactly 1 SY.

```
#[starknet::interface]
pub trait IPT<TContractState> {
    // ERC20 standard functions (inherited)

    fn expiry(self: @TContractState) -> u64;
    fn is_expired(self: @TContractState) -> bool;
    fn sy(self: @TContractState) -> ContractAddress;
    fn yt(self: @TContractState) -> ContractAddress;

    // Only callable by YT contract
    fn mint(ref self: TContractState, to: ContractAddress, amount: u256);
    fn burn(ref self: TContractState, from: ContractAddress, amount: u256);
}
```

Parameter specification.

Parameter	Type	Description
`expiry` return	u64	Unix timestamp (seconds) of maturity
`is_expired` return	bool	True if current time $\geq$ expiry
`amount` (mint/burn)	u256	PT amount (WAD-scaled)

Access control. `mint` and `burn` are restricted to the YT contract address. Calls from any other address revert with "unauthorized".

C.4 YT Interface

Semantic invariants.

- YT supply always equals PT supply.
- The global index I_{global} only increases (watermark protection).
- Interest is distributed proportionally based on YT balance and index delta.
- After expiry, YT stops accruing yield and becomes non-transferable.

Interest accounting model.

- Interest accrues via a global monotonic index updated from the oracle.
- Each user has a snapshot index I_{user} recording their last claim.
- Claimable interest: $B_{YT} \times (I_{global} - I_{user})/I_{user}$.
- Index updates occur on all state-changing operations.

```
#[starknet::interface]
pub trait IYT<TContractState> {
    // ERC20 standard functions (inherited)

    // Minting (splits SY into PT + YT)
    fn mint_py(
        ref self: TContractState,
        receiver: ContractAddress,
        amount_sy_to_mint: u256
    ) -> (u256, u256);  // returns (pt_amount, yt_amount)

    // Redemption (recombines PT + YT into SY)
    fn redeem_py(
        ref self: TContractState,
        receiver: ContractAddress,
        amount_py_to_redeem: u256
    ) -> u256;  // returns sy_amount

    fn redeem_py_and_underlying(
        ref self: TContractState,
        receiver: ContractAddress,
```

```
        amount_py_to_redeem: u256,
        token_out: ContractAddress
    ) -> (u256, u256);  // returns (sy_redeemed, underlying_out)

    // Interest
    fn claim_interest(ref self: TContractState, user: ContractAddress) ->
   u256;
    fn get_user_interest(self: @TContractState, user: ContractAddress) ->
   u256;

    // State
    fn expiry(self: @TContractState) -> u64;
    fn is_expired(self: @TContractState) -> bool;
    fn py_index_stored(self: @TContractState) -> u256;
    fn sy(self: @TContractState) -> ContractAddress;
    fn pt(self: @TContractState) -> ContractAddress;
}
```

Parameter specification.

Parameter	Type	Description
`amount_sy_to_mint`	u256	SY to split into PT+YT (WAD-scaled)
`amount_py_to_redeem`	u256	PT+YT pairs to recombine (WAD-scaled)
`py_index_stored` return	u256	Current global index (WAD-scaled)
`claim_interest` return	u256	SY amount transferred to user

Notes.

- `mint_py` requires prior SY approval; transfers SY from caller and mints equal PT and YT to receiver.
- `redeem_py` requires caller to hold equal amounts of PT and YT; burns both and returns SY.
- `claim_interest` updates I_{user} to I_{global} after transfer; subsequent calls return 0 until new yield accrues.
- After expiry, `redeem_pt_after_expiry` (not shown) allows PT-only redemption at 1:1.

C.5 Market Interface

Semantic invariants.

- Pool proportions remain within configured bounds; trades violating bounds revert.
- LP tokens represent pro-rata ownership of pool reserves.
- Swap prices follow the logit-based AMM curve (see Section 4).
- Fees accrue to the pool and are distributed to LPs on withdrawal.

```
#[starknet::interface]
pub trait IMarket<TContractState> {
    // ERC20 for LP tokens (inherited)

    // Swaps
    fn swap_exact_pt_for_sy(
        ref self: TContractState,
        receiver: ContractAddress,
        exact_pt_in: u256,
        min_sy_out: u256
    ) -> u256;  // returns sy_out

    fn swap_exact_sy_for_pt(
        ref self: TContractState,
        receiver: ContractAddress,
        exact_sy_in: u256,
        min_pt_out: u256
    ) -> u256;  // returns pt_out

    // Liquidity
    fn add_liquidity(
        ref self: TContractState,
        receiver: ContractAddress,
        sy_desired: u256,
        pt_desired: u256,
        min_lp_out: u256
    ) -> (u256, u256, u256);  // returns (sy_used, pt_used, lp_minted)

    fn remove_liquidity(
        ref self: TContractState,
        receiver: ContractAddress,
```

```
        lp_to_burn: u256,
        min_sy_out: u256,
        min_pt_out: u256
    ) -> (u256, u256);  // returns (sy_out, pt_out)

    // State
    fn get_reserves(self: @TContractState) -> (u256, u256);
    fn get_ln_implied_rate(self: @TContractState) -> u256;
    fn expiry(self: @TContractState) -> u64;
    fn pt(self: @TContractState) -> ContractAddress;
    fn sy(self: @TContractState) -> ContractAddress;
}
```

Parameter specification.

Parameter	Type	Description
`exact_pt_in / exact_sy_in`	u256	Exact input amount (WAD-scaled)
`min_sy_out / min_pt_out`	u256	Minimum output; reverts if not met
`sy_desired / pt_desired`	u256	Maximum amounts to deposit
`min_lp_out`	u256	Minimum LP tokens; reverts if not met
`get_reserves` return	(u256, u256)	Current (SY, PT) reserves
`get_ln_implied_rate` return	u256	$\ln(1 + r)$ scaled by WAD

Pricing reference. All swap prices follow the logit-based AMM defined in Section 4. The exchange rate is computed as:

$$\text{rate} = \frac{\ln(p/(1-p))}{r_{scalar}} + r_{anchor}$$

where p is the PT proportion and r_{scalar} decays toward expiry.

Slippage protection. All swap and liquidity functions include `min_*_out` parameters. If the computed output is less than the minimum, the transaction reverts. This protects users against price movement between transaction submission and execution.

C.6 Revert Conditions Summary

Condition	**Affected Functions**
Output below minimum	All swaps, deposits, withdrawals
Insufficient balance	All transfers, burns, redemptions
Unauthorized caller	PT mint/burn (non-YT caller)
Proportion bounds violated	Market swaps
Zero amount	All state-changing functions
Expired (where applicable)	`mint_py`, `redeem_py`
Not expired (where applicable)	`redeem_pt_after_expiry`

D Glossary

Glossary

This glossary defines key terms used throughout this specification. Terms are ordered alphabetically.

Term	Definition
AMM	Automated Market Maker; a smart contract that provides liquidity and determines prices algorithmically based on pool reserves and a pricing function.
cairo-fp	Cairo fixed-point arithmetic library using 64.64 format (64 integer bits, 64 fractional bits), enabling high-precision transcendental function computation.
Expiry (T)	Unix timestamp (seconds) at which PT becomes redeemable for exactly 1 SY and YT stops accruing yield.
Implied Rate (r)	The annualized yield rate inferred from the current PT/SY exchange rate, assuming PT converges to 1 SY at expiry. Computed as $r = -\ln(P_{PT})/\tau_y$.
Logit	Transformation $\ln(p/(1-p))$ mapping pool proportions in $(0, 1)$ to the real line $(-\infty, +\infty)$, used to derive the AMM exchange rate.
LP Token	Token representing proportional ownership of a PT/SY liquidity pool; entitles holder to pro-rata share of reserves and accumulated fees.
Market	Smart contract implementing the PT/SY AMM, responsible for swaps, liquidity provision, and price discovery.
Minimum Liquidity	Small amount of LP tokens (typically 10^3) permanently locked on first deposit to prevent share inflation attacks.
PT (Principal Token)	Token representing a claim on principal that redeems for exactly 1 unit of SY at or after expiry. Trades at a discount before expiry reflecting the implied yield.
PY	A matched pair of PT and YT tokens (1 PY = 1 PT + 1 YT), which together are economically equivalent to 1 SY before expiry.
PY Index (I_{global})	Monotonically non-decreasing cumulative index tracking yield accrual for YT holders. Updated from oracle on state-changing operations.
Rate Anchor (r_{anchor})	Dynamic offset parameter that ensures implied yield continuity across successive trades; updated after each swap to preserve rate consistency.
Rate Scalar (r_{scalar})	Time-dependent scaling parameter that controls AMM price sensitivity to changes in pool composition. Increases toward infinity as expiry

Symbol Reference

Symbol	Name	Units
P_{PT}	PT price in SY terms	Dimensionless (0 to 1)
P_{YT}	YT price in SY terms	Dimensionless (0 to 1)
r	Implied annual rate	Per-year (dimensionless)
τ	Time to expiry	Seconds
τ_y	Time to expiry (annualized)	Years
T	Expiry timestamp	Unix seconds
Y	Seconds per year	31,536,000
I_{global}	Global yield index	WAD-scaled ratio
I_{user}	User yield index snapshot	WAD-scaled ratio
r_{scalar}	Rate scalar parameter	WAD-scaled
r_{anchor}	Rate anchor parameter	WAD-scaled
p	PT proportion in pool	Dimensionless (0 to 1)

End of Specification

Document Version: v1.0
Last Updated: December 2025
License: BSL-1.1 (converts to GPL-3.0 on 2028-12-19)

This document is a technical specification and does not constitute investment, legal, or financial advice. Users should conduct independent analysis before interacting with the protocol.